



学 校 代 码 10459

学号或申请号 Z201217281

密 级

郑 州 大 学

工程硕士专业学位论文

Redis 缓存技术研究及应用

作 者 姓 名: 邱书洋

导 师 姓 名: 王黎明

专业学位名称: 软件工程

培 养 院 系: 信息工程学院

完 成 时 间: 2015 年 11 月

A dissertation submitted to
Zhengzhou University
for the degree of Master

Research and Application of Redis Caching Technology

By Shuyang Qiu
Supervisor: Prof.LimingWang
Software Engineering
School of Information Engineering
November, 2015

摘要

随着计算机和网络技术的不断发展与推广，网络应用信息量和用户数量的激增给互联网应用公司和互联网运营商带来了巨大压力，如何提升网络应用服务质量，如何从根本上提升网络应用的实时性和并发性，已经成为亟待解决的关键难题。

应用层缓存技术的出现为解决网络应用难题指明了方向，缓存模块设计的好坏直接影响着网络应用的客户体验，Redis 内存数据库缓存技术应用场景广泛、具备理想的读写速度和丰富的数据结构模型，是最为流行的缓存技术之一。

本文以 Redis 缓存技术为研究对象，以提升“互联网金融服务平台”高可用性为研究抓手，首先通过对缓存技术分类和缓存技术产品介绍，引出 Redis 缓存技术的定义，它是一款开源的 Key-Value 模型内存数据库，其工作原理是将热点数据常驻在内存中，并在内存中完成对数据的操作，保证了数据的处理速度。其次在 Redis 缓存技术的定义的基础上，本文全面介绍了 Redis 缓存技术的基础特性和常用应用场景，通过同类产品技术性能参数对比，分析 Redis 缓存技术的优势和不足，并通过对其不足之处的研究，找出弥补措施，提供 Redis 的完美解决方案。最后在 Redis 缓存技术研究的基础上，将 Redis 缓存技术应用于“互联网金融服务平台”并完成对平台技术架构的缓存模块设计及关键业务流程技术改造，满足了“互联网金融服务平台”业务实时性强和高并发的要求，实现系统性能大幅提升的既定目标。

关键词：Cache；数据库；Redis；互联网金融服务平台

Abstract

With the development of computer and network technology, the number of network application users is getting increased a lot, which brought enormous pressure to the Internet operators and Internet applications companies. How to improve the network quality of the application and how to fundamentally enhance the real-time and concurrent network application has become the key problem to be solved.

Application layer cache technology can solve the problem of network applications. The cache module design quality directly affects the network application of customer experience. The Redis cache technology application scenarios, with ideal of the read and write speed and rich data structure model, is one of the most popular cache technology.

As the research object, the Redis cache technology is in order to enhance the "Internet Financial Services Platform" high availability. First of all, according to the introduction of cache technology classification and cache technology products, it leads to the definition of Redis cache technology. It is a source of key model in the memory database. Its working principle is the hot data resident in memory and in memory complete the operation of the data, to ensure that the speed of data processing. Secondly, on the definition of Redis cache technology based, the paper gives a comprehensive introduction to the Redis cache technology characteristics and application of common scene. The paper has researched the performance parameters for the similar products of Redis, which is showing the advantages and disadvantages for Redis cache technology. For the shortcomings of the research it has found the solutions. Finally, based on the research of the Redis cache technology, Redis cache technology can be applied in the "Internet Financial Services platform". It completed the technical platform architecture cache module design and key business process transformation, which met the "Internet Financial Services Platform" real-time business concurrent. It significantly enhanced the system performance.

Key words: Cache; Database; Redis; Internet financial Service Platform

目录

摘 要.....	I
ABSTRACT.....	II
1 绪论.....	1
1.1 研究背景.....	1
1.2 研究目的和意义.....	2
1.3 国内外研究现状及发展趋势.....	2
1.4 研究内容及结构.....	3
2 关键理论与技术.....	4
2.1 Cache 的定位.....	4
2.1.1 不同时代对 Cache 的理解.....	4
2.1.2 Web 应用下 Cache 的分类.....	5
2.2 主流 Cache 技术概述.....	11
2.3 本章小结.....	15
3 Redis 缓存技术研究.....	16
3.1 Redis 概述.....	16
3.2 Redis 基础特性.....	17
3.2.1 Redis 数据结构.....	17
3.2.2 Redis 持久化.....	23
3.2.3 Redis 主从复制.....	25
3.2.4 Redis 事务处理.....	26
3.2.5 Redis 常用应用场景.....	28
3.2.6 Redis 的注意事项.....	30

3.3 性能对比及问题改进.....	33
3.3.1 同类产品对比.....	33
3.3.2 问题改进.....	35
3.4 本章小结.....	39
4 Redis 在互联网金融服务平台的应用.....	41
4.1 系统概述.....	41
4.1.1 平台总体架构.....	41
4.1.2 平台功能介绍.....	44
4.1.3 重点流程剖析.....	46
4.1.4 系统瓶颈.....	49
4.2 Redis 在缓存架构设计中的应用.....	50
4.2.1 系统缓存架构设计.....	51
4.2.2 系统持久化设计.....	52
4.2.3 高可用设计.....	53
4.2.4 容量管理设计.....	55
4.2.5 Redis 安装及配置.....	55
4.2.6 优化后的应用流程场景.....	59
4.3 本章小结.....	61
5 测试及效果分析.....	62
5.1 性能测试.....	62
5.2 效果分析.....	65
5.3 本章小结.....	67
6 总结与展望.....	68
6.1 工作总结.....	68
6.2 工作展望.....	68
参考文献.....	70

致 谢.....	73
个人简历.....	74
攻读学位期间发表的学术论文.....	75

1 绪论

1.1 研究背景

随着科学技术的不断发展进步，如今的时代可以说是属于计算机技术的时代^[1]，互联网不仅已经与人们的日常生活息息相关，而且在各个领域都有着飞速发展的势头。云计算、物联网等新兴技术的出现，催生了互联网金融的产生，互联网金融的出现大力冲击和影响了传统金融行业。自 2013 年 7 月 20 日起，中国人民银行决定全面放开金融机构贷款利率管制^[2]，近年来利率市场化的步伐不断加速，直到 2015 年 10 月 24 日起，中国人民银行决定对商业银行和农村合作金融机构等不再设置存款利率浮动上限^[3]，在利率市场化的大背景下，传统银行不仅要承受互联网金融企业的强烈冲击，还要背负着非金融企业带来的跨行业瓜分的压力，要想在如此激烈的竞争中不被淘汰或吞并，就必须在深刻理解互联网金融本质的基础上，利用互联网强化自己的工具，建立高效、快捷、安全的互联网金融服务体系。

2015 年可以说是互联网年。2015 年 3 月 5 日，第十二届全国人民代表大会第三次会议上，李克强总理提出指定了“互联网+”行动计划并在政府工作报告中提出指定“互联网+”行动计划，是要促进互联网和传统行业的结合^[4]。“互联网+”的成功案例已经数不胜数：想逛街上淘宝，想打车叫滴滴，买家电上京东，买理财上支付宝；2015 年 9 月 23 日，习近平主席出席了第八届“中美互联网论坛”在西雅图的开幕式并发表讲话强调：当今时代，社会信息化迅速发展。从老百姓衣食住行到国家重要基础设施安全，互联网无处不在。一个安全、稳定、繁荣的网络空间，对一国乃至世界和平与发展越来越具有重大意义。如何治理互联网、用好互联网是各国都关注、研究、投入的大问题。没有人能置身事外。由此可以看出，互联网在我们的生活中肩负着越来越重大的使命，互联网的发展不仅推动传统行业的转型，更是促进民生的法宝^[5]。

传统意义上的 Cache 就是硬件服务器的存储子系统，负责着匹配 CPU 的计算速度和读写主硬盘速度的差异^[6]。互联网时代的缓存技术是解决互联网应用面对实时性、并发访问量、数据量三个方面都要求极其苛刻的背景下的关键技术，不过对于城商行互联网金融初期，依然摆脱不了传统银行系统架构设计的束缚，

坚持了保守的态度和思想，抵制了新技术的尝试和使用，缓存的设计考虑非常之欠缺，导致了后期支撑能力的明显滑坡。

我们通过对 Redis 缓存技术的研究结合对互联网金融服务平台架构的分析，提出是否能用 Redis 技术对平台的进行技术改造的想法。

1.2 研究目的和意义

对于传统金融行业来说，尤其是在利率市场化后银行利差收入逐步缩水和当前经济大背景不景气的状况下，面临互联网金融的强烈冲击，更是雪上加霜，为了生存的需要，城商行都在打造自己的互联网金融服务平台，服务平台要求实时性高、响应快、数据量大的特点。为了摆脱传统关系数据库模式的设计思路对互联网金融平台的拖累，我们研究 Redis 缓存技术特点，通过 Redis 缓存技术的应用切实提升互联网金融服务平台的服务水平，同时保证系统的安全可靠。

1.3 国内外研究现状及发展趋势

国外关于缓存技术相关研究与应用早于国内很多年，国外有很多很有名且很好用的缓存技术产品，例如 Nginx，属于 HTTP 和方向代理服务器，是由俄罗斯某站点开发，2004 年发布了第一版本，其特点在于稳定性高、系统资源消耗低，俄罗斯国内各大型的入口网站都在使用该技术，在美国很多虚拟主机行业也在使用该技术，其并发能力强的特点在同类技术中表现突出，后被我国国内各大型网站都采用，如：新浪、百度等；另外还有一项明星技术产品是 Squid，可以作为网页服务器的缓存代理向服务器发送请求和缓存数据，也可以使局域网用户使用代理上网，主要运行于 Linux 操作系统^[7]；美国很有名的维基媒体系统架构内使用了 EHcache，很大程度上提高了系统的性能。

内存数据库也是缓存技术的一种，内存数据库的雏形是在 1969 年由 IBM 公司开发研制的基于层次模型的数据库管理系统 IMS，IMS 系统的设计包含基于内存的数据管理方法。内存数据库技术的进一步发展和成熟是在 1984 年，当年 D J DeWitt 等人发表了一篇名为《主存数据库系统的实现技术》一书，文中第一次提出了 MainMemoryDatabase 的概念^[8]。1994 年以来，伴随着互联网行业应用的飞速发展，越来越多基于网络的应用都拥有了一致的要求：支持大并发量访

问和高速响应，有了这些应用的需求背景，内存数据库进入了产品发展期和市场成熟期。与此同时，半导体技术也正在飞速发展，半导体内存被大规模生产，并且容量越来越大，价格确越来越低，应用需求和技术进步两个方面在促使内存数据库技术由理论转化为现实^[9]。

内存数据库的出现得到了广大数据库爱好者的追捧和研究者的广泛关注，经过众人几十年来的努力，内存数据库研究在数据结构、事务处理、数据持久化、并发处理、数据主从复制等方面获得了重大突破。

目前，内存数据库的主要研究方向还是内存数据库的实际应用。理论层面的研究包含了数据的存储、索引、恢复、事务控制等内容；应用的研究主要考虑互联网应用如何引入内存数据库的使用，比如互联网购票系统^[10]、电信计费系统^[11]、车联网^[12]等。对缓存技术的理论和技术研究，主要在第3章节里进行详细介绍。

自1994年内存数据库技术得到长足发展以来，市场上推出了越来越多的商品化或者开源产品^[13]，例如：94年美国OSE公司的商业化产品Polyhedra，01年加拿大Empress公司推出EmpressDB。内存数据库产品由于具有鲜明的技术特征，因此在基于互联网要求高并发和高速响应能力的应用领域得到了广泛应用。互联网金融时代，对银行互联网金融服务平台建设提出了更高的要求，我们通过各种方式来尝试如何提升服务平台的性能。

1.4 研究内容及结构

本文通过6个章节展开论述，文章结构及每章概要内容如下：

第1章：主要阐述课题研究的背景、目的及意义、国内外研究现状以及未来缓存技术的发展趋势。

第2章：主要介绍缓存关键的理论和技术，包含缓存技术的定位和开源的缓存技术进行研究。

第3章：重点介绍论文的主题技术：redis缓存技术，深度剖析Redis缓存技术的基础特性和实现原理，通过性能测试和同类产品对比体现Redis缓存技术的特长和优势。

第 4 章：通过互联网金融服务平台技术架构分析和剖析重要业务流程，找出制约系统服务能力的瓶颈问题，进而利用 Redis 缓存技术改进现有系统架构及业务流程，这也是本文的关键内容，最后实现对平台的持久化、高可用等设计。

第 5 章：对改进后的互联网金融服务平台主要业务流程进行性能验证测试，测试结果表明，Redis 的应用对平台性能有了大幅的提升。

第 6 章：总结与展望。

2 关键理论与技术

Cache 是一个智能化的产品，当 html 页面、JS 脚本、图片等资源被首次访问时会以副本的形式保存在服务器与客户端之间，当用户访问请求同一个资源信息时，Cache 会自动根据相应机制判定资源是否更新，若无更新，则直接读取本地信息返回客户端，若服务端标明资源有更新时，先从服务器端更新资源后返回客户端，通过这种减少客户端与服务器端访问次数的方法从而达到节约带宽和提高响应速度的目的。Cache 的主要作用表现为减少网络资源消耗、减少网络延迟、减小服务器的请求压力、增强系统可用性。下面我们将对 Cache 的概念和不同节点的 Cache 的部署做介绍和对比。

2.1 Cache 的定位

Cache 是一个比较笼统的概念，就是数据交换的缓冲区，这种缓存区的技术实现既可以通过硬件(硬盘和内存中间增加一个临时存储器)，也可以通过软件的形式(软件架构中增加缓冲区模块，对用户请求做分析)。这一节中我们主要来区分传统的 Cache 和 Web 缓存的不同意义，以及不同部署下的 Cache 分类。

2.1.1 不同时代对 Cache 的理解

传统意义的 Cache 是在计算机存储系统层次结构中，介于中央处理器(CPU)和主存储器之间的高速小容量存储器，属于 CPU 的一部分，是数据交换的缓冲区，当某硬件发出读取数据的指令时，系统会首先在缓存区查找需要的数据内容，找到则直接执行返回，否则就从主存储器中找，由于 CPU 与主存储器之间的速度差异至少是 20 倍以上，所以 Cache 就是为了解决 CPU 速度和内存速度的速度差异问题^[14]。

现代意义的 Cache 是互联网各大服务器端模仿硬件 Cache 的工作原理，在服务端架构里增加 Cache 模块设计，使用各种 Cache 技术或者内存数据库，实现客户端在访问服务器资源时，先查询 Cache 模块，存在则直接响应客户端请求，否则将客户端请求转发后台服务器，在从资源库获取客户端请求内容。由于 Cache 模块的增加，可以让客户端的响应速度更快，直接提升客户对网站速

度快的友好体验，同时缓存文件可以反复使用，可以节省不少带宽资源，为用户节省流量。Cache 定位如图 2.1 所示：

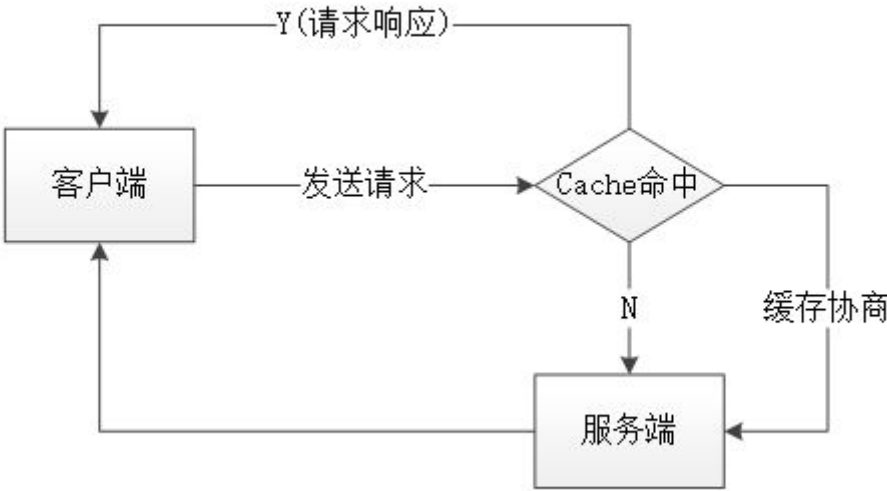


图 2.1 Cache 定位图

2.1.2 Web 应用下 Cache 的分类

Web 应用程序是支持多用户同时访问的，那么其站点服务器就可能在某一个时间点达到一个访问量并发的最大值，使得站点再次被更多用户访问的时候，服务器的相应速度会由于负载能力有限的原因而变慢甚至服务器崩溃。面对这种场景的考虑，为了更好的提升服务质量，预防服务器并发访问达到峰值的情况出现，站点管理人员必然要想办法应付这些情况，那么是不是可以考虑追加投资提升服务器硬件配置？或者网络部署考虑负载均衡或者提升带宽流量？然而这些手段可以说在一定程度或一定时期内能缓解服务器压力，但是不能从根本上解决服务器慢的结果，所以硬件的提升不能完全的解决问题，此时考虑缓存不得不说是个不错的想法。

我们先来分析一下 Web 请求流转的路线，当客户通过浏览器发送一个网站首页的请求时，请求从客户端发往服务器端，而客户端和服务端可能存在某些代理服务器或转换服务器，那么我们就考虑至少可以从三个位置着手设置缓存：客户端、代理端、服务器端。下面我们根据缓存位置的不同，对缓存类别做如下介绍：

(1) 客户端缓存

所谓的客户端也就是浏览器缓存机制^[15]，即根据与服务器端约定好的一套规则工作:在同一会话连接过程中，使用本地硬盘存储或浏览器内存上的副本数据完成高速响应的任务，例如在浏览网页过程中，采用前进或后退的方式访问了相同的数据内容，由于数据内容一致，所以响应很快，因为他拒绝了与外界网络和服务器的数据交互。客户器缓存模型如图 2.2 所示：

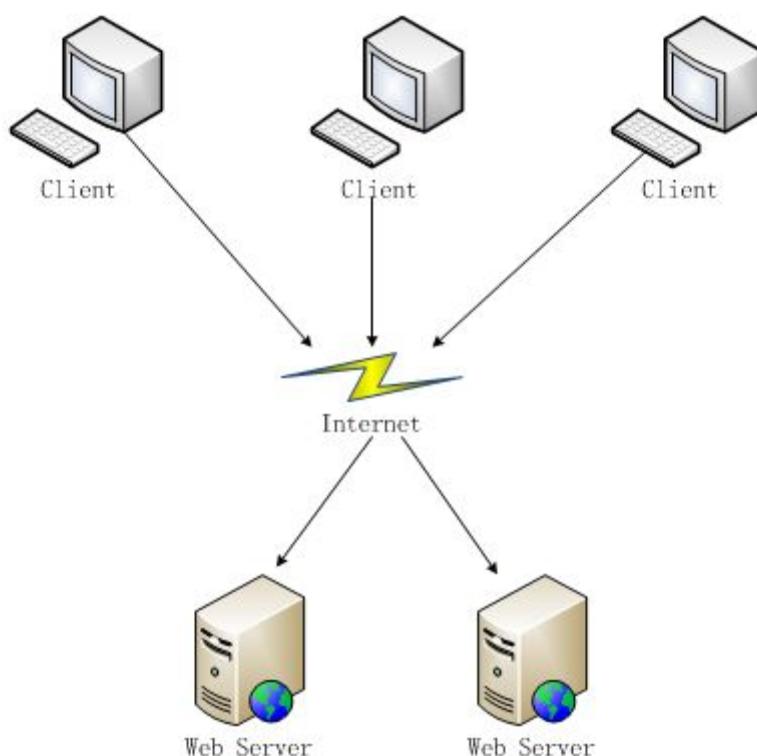


图 2.2 客户端缓存

缓存位与每个客户端 Client 的本地硬盘内或本地浏览器的内存中，这样优势很明显：

访问容易且高效，避免了其他资源消耗，但劣势也是显而易见的：数据资源只能存在于一个 Client 的本地硬盘内，独立于浏览器而存在，因此不能被更多客户端共享。

(2) 代理缓存

为了弥补客户端服务器的资源不能共享的缺点，我们考虑在客户端与 Internet 中间通过部署一套服务器的方式来完成资源共享，我们暂且称之为中转

服务器，其缓存模型如下图所示：当 A 客户端向源服务器发送资源请求时，请求先发往中间服务器，由中间服务器处理请求信息后，转发到源服务器端，相当于代理客户端发送请求，这样在源服务器返回请求资源时，中间服务器会把相应资源信息返回给客户端 A 同时保留一个该请求的备份，那么在客户端 B 或者 C 请求同一个源服务器的资源时，首先发往中间服务器，中间服务器根据相应规则与访问控制列表去匹配，若匹配成功，则无需访问源服务器而能直接并快速响应客户端请求，我们称这种中间服务器模式为代理缓存^[16]。模式如图 2.3 所示：

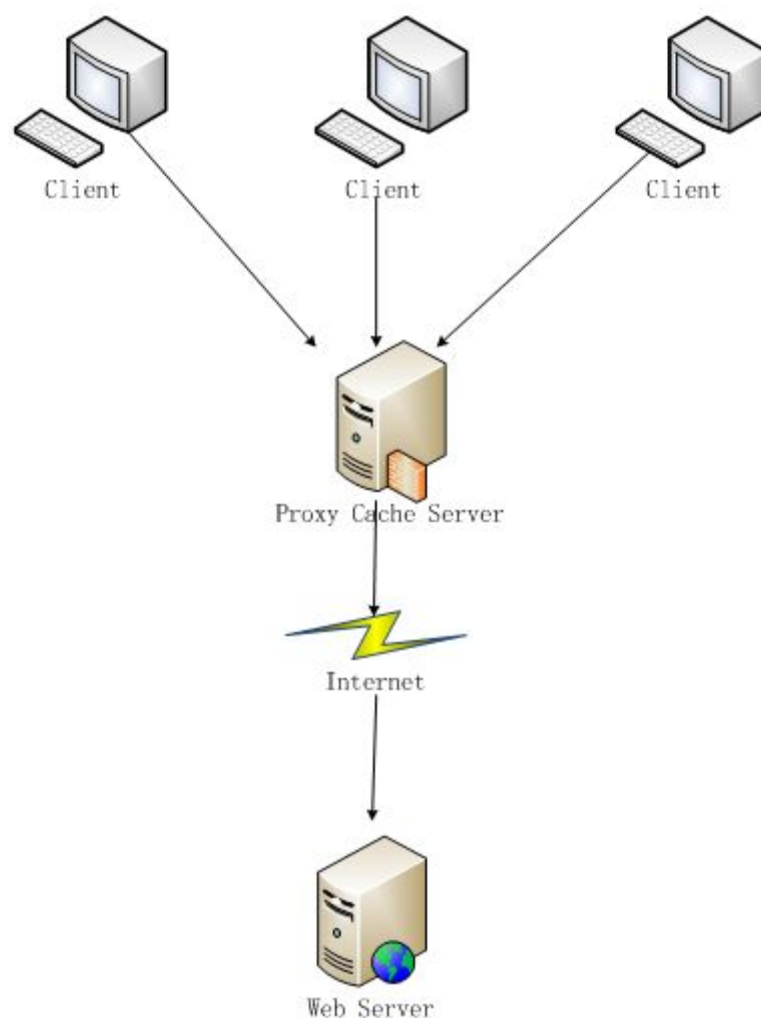


图 2.3 代理缓存

代理缓存通常位于网络的网关附近，这样可以减少网络的交互从而降低对带宽的要求。当网络访问量很大或者并发很高的时候，单个代理缓存服务器也会存在很大的压力，那么我们可以考虑通过部署多个代理缓存服务器的模式来缓解访问压力，我们把这种模式称为缓存集群，具体模式如图 2.4 所示：

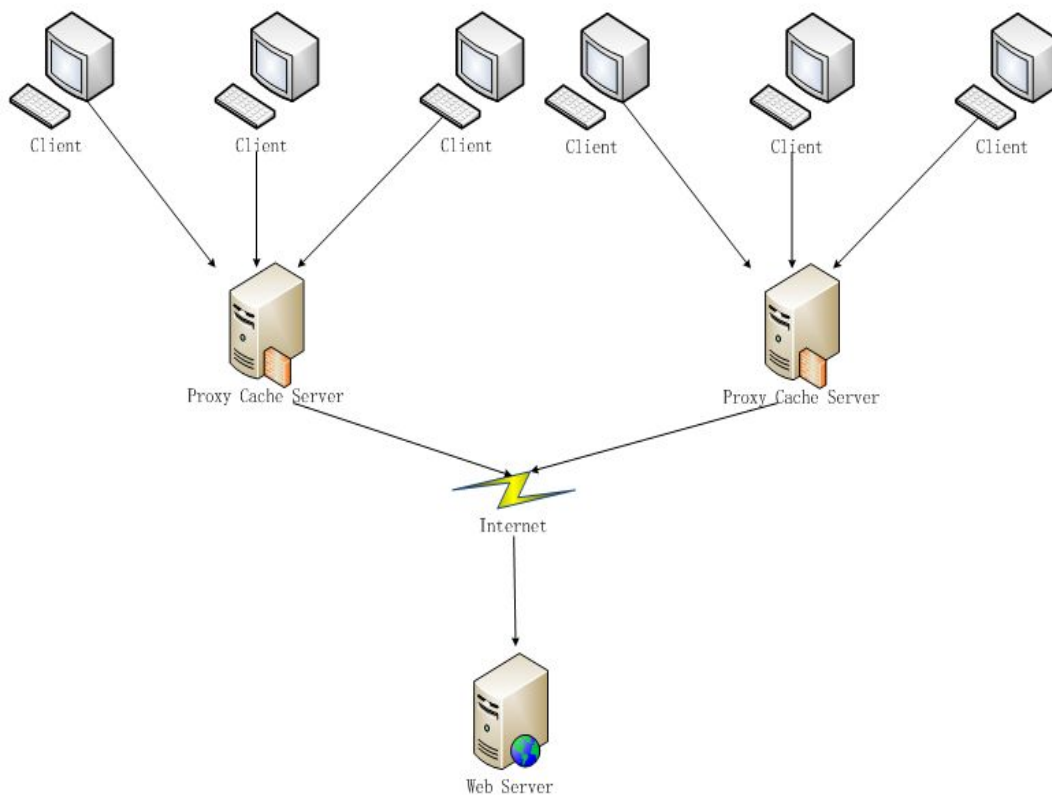


图 2.4 代理缓存集群

该模式下，代理服务器上的资源就可以被多客户端共享，同时也减少了网络通信，但是客户端需要用户专门去配置代理地址和端口信息，并且只转发他能够支持的协议数据，同时增加了部署及为后期维护代理缓存服务器的相应基础设施成本；缓存集群则更复杂，技术层面要需要考虑多个代理服务器之间的资源同步等问题。

（3）反向代理缓存

从上面的分析我们可以看出，代理缓存服务器其实就是与客户端一体的，那么根据这个思路，是不是可以在服务器端部署一个代理与服务器属于一体的呢？没错，部署在源服务器前的缓存服务器，我们称之为反向代理，他的主要用处是不在让客户端与源服务器直接通讯，他对于客户端来说才是可见的源

服务器，当收到客户端发来的请求时，反向代理服务器会向其后面对应的一台或多台负载应用服务器，根据各服务器负载情况，动态的将请求发给合适的负载服务器，以减少源服务器负载、保障源服务器的安全，同时还能完成源服务器的负载均衡，在整个请求传输过程中，反向代理服务器可以理解为一个配备有大容量内存和高速磁盘的智能路由器，负责匹配请求信息，若请求信息为静态数据时，则可直接返回，若请求信息含有动态信息，则需要保持客户请求会话的同时，智能获取动态信息后返回客户请求^[17]。其模式如图 2.5 所示：

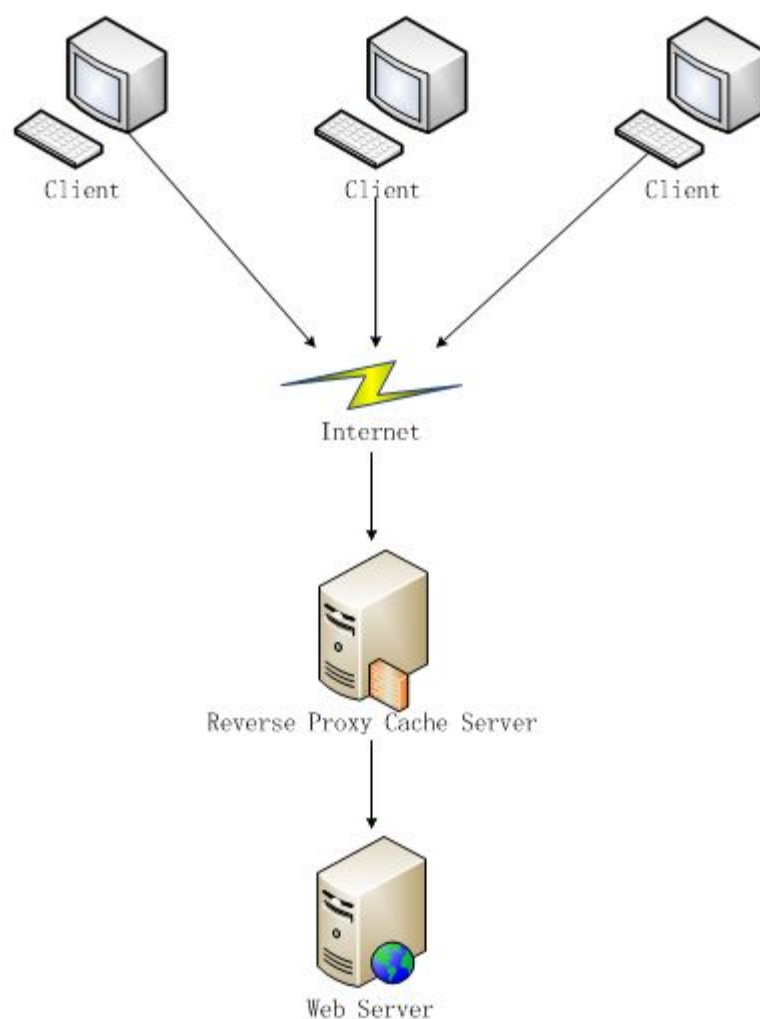


图 2.5 反向代理缓存

这种部署模式，使得客户的静态请求很快很容易的被响应，并且实现了服务器的负载均衡，但是也可能导致额外的网络通讯，特别是请求含有动态数据时，反向代理要维持两个方向的会话的连接，会造成一定的网络资源消耗。

（4）Web 服务器缓存

以上的各种代理都是在请求链路上通过硬件部署的方式来减少客户端与服务器的请求交互次数，从而达到高效和稳定的目的，而 Web 服务器缓存（模式如图 2.6 所示），则是在 Web 服务器端想办法，就是在根本上解决问题，那我们简单分析下，Web 服务器端实现缓存的手法有哪些：首先可以考虑系统层面处理，通过 Cache 库，对对常会且耗费资源的数据缓存起来，不过主要更新资源费点功夫；其次考虑从应用层着手，通过代码逻辑和缓存策略实现对数据的内存存储，减少数据的读写处理，提升服务相应速度；最后我们还可以考虑在数据库层面使用基于内存、键值对的存储数据库。

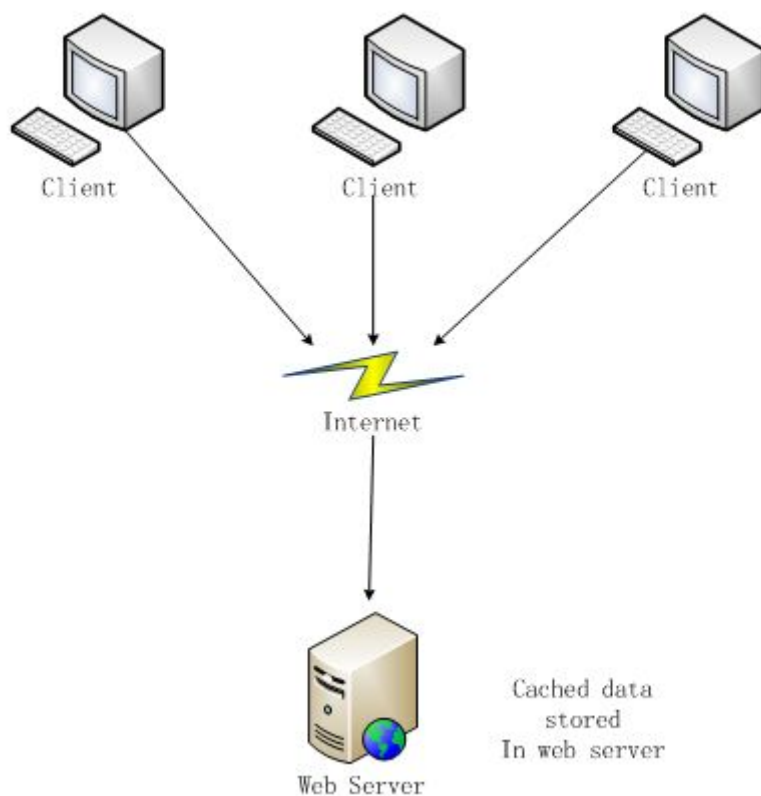


图 2.6 Web 服务器缓存

Web 服务器缓存如果各方面缓存都很到位的话，则系统架构会较为复杂，管理起来困难，但服务性能和质量会很棒，因为他减少了与数据库和其他服务器之间的数据检索，服务器的各个模块各尽其责。

通过以上对 Cache 的定位分析，我们可以发现客户端到服务器端整条链路上，我们有很多办法去实现缓存，但是每种缓存方法都各有优点和缺点，为了实现最大意义上的缓存处理，我们要实现全链路各个节点的缓存处理，这样每个节点缓存都会发挥自己的功能，各个节点的缓存还能互相弥补，将缓存机制的用处最大化，大大提升网络相应速度。

2.2 主流 Cache 技术概述

上文在对 Cache 做分类处理时，我们有提到关于缓存服务器设计的技术，本节中我们将对这些技术进行详述和对比，从中得出广告投放系统 Cache 设计的优势。

(1) EHCACHE

EHCACHE 是一个基于 Java 语言开发，主要是适用于进程内缓存框架的系统，有响应快、精确度高等特点，最初了解 EHCACHE 是从 Hibernate 的缓存开始的。

Ehcache 是一种最流行的开源 Java 分布式缓存。主要面向于通用缓存,Java EE 和轻量级容器这三类。它具有内存和磁盘存储,缓存加载器,缓存扩展,缓存异常处理程序,一个 gzip 缓存 servlet 过滤器,支持 REST 和 SOAP api 等特点^[18]。

下图是 Ehcache 在应用程序中的位置如图 2.7 所示：

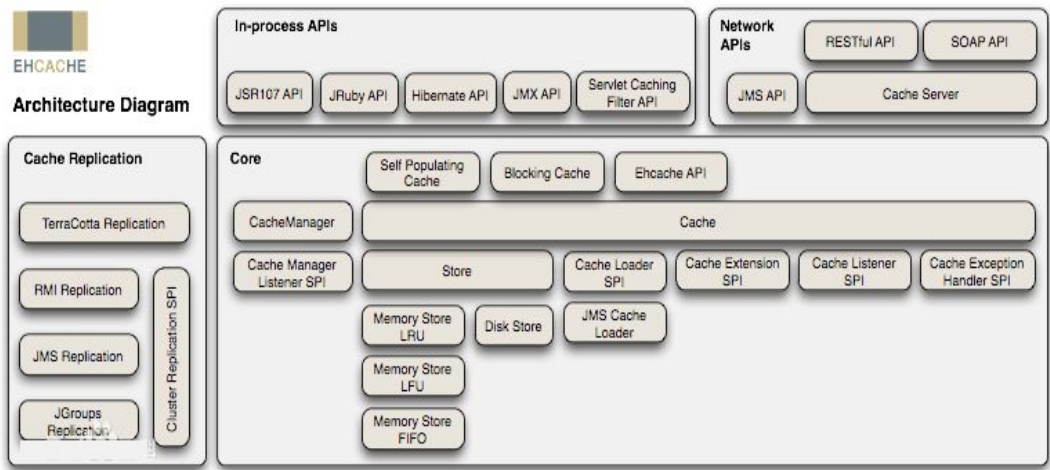


图 2.7 Ehcache 应用位置图

主要的特点是：

1. 高速响应；
2. 配置简单；
3. 结构清晰；
4. 缓存数据分为磁盘级和内存级；
5. 缓存的数据在 VM 重启时会从磁盘重新读取，方便了 EHCACHE 的使用；
6. 支持 API、RMI 等多种方式实现分布式缓存处理；
7. 提供缓存管理器的监听接口；
8. 管理模式强大：既支持单台服务器的多个缓存管理器实例也支持一个实例的多缓存区域管理，模式灵活。

（2）OSCache

OSCache 是一种基于 Web 应用的缓存机制，具有在 JSP 页面内实现快速内存缓存的功能，由 OpenSymphony 公司组织开发，属于一个高性能的 J2EE 缓存框架，可以不受限制的缓存任何对象，因此任何 Java 应用都可以考虑其作为缓存解决方案^[19]。OSCache 的存储共享资源域分为应用的全局共享域和单个请求 session 域两类，共享的资源有过期时间属性，根据过期时间，使用 flush 方式完成资源清理和重新加载的动作。

OSCache 的特点是：

1. 可以缓存任何 java 对象，缓存对象范围广，支持 JSP 页面的部分缓存也支持 HTTP 请求的全量缓存；
2. 提供全面的 API 接口供调用；
3. 缓存的永久性，缓存随时支持写入磁盘操作，因此不用担心重要的数据在缓存中由于重启等应急操作而丢失的风险；
4. 支持集群缓存并且支持单个缓存的参数设置和调整；
5. 缓存记录有时效属性，可以控制缓存的时效性，包括可插入等刷新策略。

（3）JBossCache

JBossCache 是一个树形结构、支持集群、支持事务的缓存技术。

JBossCache 是一个基于 Java 事务处理的缓存系统，主要目的是 Java 应用的框架基础之上，实现缓存的集群管理，集群间的缓存数据能自动复制，适用于服务器应用也适用于 Java SE 应用模式^[20]。

JBossCache 提供两种缓存方式：核心缓存（TreeCache）和 POJO 缓存（TreeCacheAOP）。

核心缓存也称为 TreeCache，按照功能种类不同可以划分为本地缓存、复制缓存和失效缓存三类，实现方式是把数据以 Key/Value 对的形式存储在一个树形结构的各节点上，出于可复制或持续性的要求，这些数据都被序列化了。复制缓存和失效缓存都属于分布式缓存，分布式又分为同步和异步，同步顾名思义是缓存数据会实时的复制到集群的其他缓存中，这样实时性高，但是对于正在复制中的缓存，当有请求调用时，则会因为正在复制而导致调用阻塞，直到复制结束后才能正常提供服务；异步是针对同步而言，原理类似，不过不用请求不会因为复制而阻塞。

POJO 缓存也叫做 TreeCacheAOP，机制相对复杂，是利用字节码编织的原理来内省（introspecting）用户类，并在用户类的各个域配置侦听器功能，一旦被侦听的域值发生变化，侦听会立刻将变更信息反馈给缓存，缓存只复制发生变化的域值信息而不会去复制整个用户类，这种细粒度复制机制保持了缓存的高效能力。当缓存面对结构复杂、数据量庞大的对象时，细粒度复制更有助于提高性能，反之对于结构简单的对象，例如只有几个变量值或者一个简单的类，则 POJO 缓存则没有什么发挥空间，对于提高性能方面也没有价值，更甚者说不如是浪费资源。

（4）MemCached

Memcached 是由 Danga Interactive 公司开发的一款高性能的分布式内存对象缓存软件，原理是将数据和对象缓存在内存中，当有来自应用的数据对象访问请求时，直接从内存中返回请求信息，减少了应用与数据库的交互，从而达到提升 Web 动态应用的响应速度和提高可扩展的空间的目标^[21]。Memcached 基于一个存储 Key/Value 对的哈希表。其守护进程是用 C 语言编写，但是对客户端的编写语言没有特定要求，支持多种编程语言，客户端通过 memcached 协议与底层守护进程通信。其运行模式如图 2.8 所示。

Memcached 的特性为：

（1）基于 C/S 架构，协议简单：服务器端启动 Memcached 进程后，客户端可以通过 telnet 方式或者通过编程语言方法与数据库完成通讯，实现数据的存储和查询，服务端与客户端的通讯报文无需复杂的 XML 格式，而是基于简单的文本行协议即可；

(2) 基于 libevent 的事件处理：libevent 是一个集成了多种操作系统事件处理功能的封装接口，能够支持例如 Linux、BSD 等操作系统，API 接口包含各操作系统的查询函数，Memcached 使用 libevent 事件处理来进行网络并发请求的处理，能够在高并发连接的场景下，保持系统的高速响应能力；

(3) 内置内存存储方式：默认使用 Slab Allocation 机制来分配和管理内存，基本原理是按照预先规定的大小，将分配的内存切割成特定长度的块，已解决内存碎片问题，从而实现内存的重复使用；

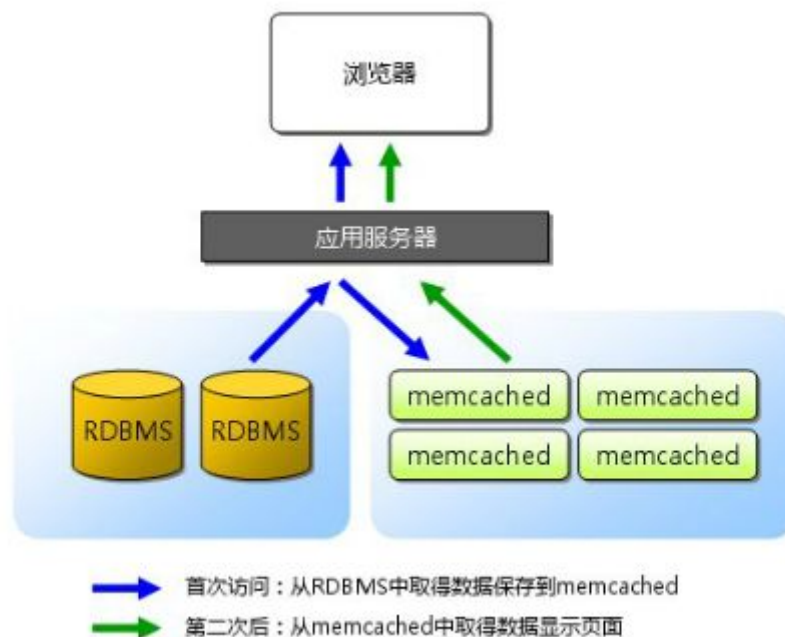


图 2.8 MemCached 运行模式图

(4) 基于客户端的 Memcached 分布式。

(5) Memcached 的过期处理机制：Memcached 本身不具备监听记录是否已过期的功能，而是通过获取数据记录的时间戳信息，比对当前时间戳后来判定记录是否过期，这种被动的技术被称之为 lazyexpiration，所以，Memcached 不会在过期监控上耗费机器资源。Memcached 使用的过期算法是 LRU (Least Recently Used)，在具体处理时会优先使用已超时数据的空间，但是在记录新增的过程中，仍然会有空间不足的情况出现，因此针对这种问题，就会删除“最近最少使用”的记录，把空间腾出分配给新增记录，是实用角度考虑，这个机制也是十分理想。

2.3 本章小结

本章介绍了 Cache 技术的相关理论知识，主要包括 Cache 的定义、分类以及当前最常用的几个开源的 Cache 技术介绍，同时对开源的 Cache 技术通过实现难度、适用范围和性能提升幅度等方面做了对比，以体现各技术的优势和劣势。

3 Redis 缓存技术研究

Redis 是一个开源、支持网络、基于内存、键值对存储数据库，适用 ANSI-C 编写，是目前最流行的键值对存储数据库之一，也是本文重点研究的缓存技术，本章我们将对 Redis 缓存技术进行一个全面彻底的研究和解析。

3.1 Redis 概述

Redis 是一个开源的、基于内存或持久化、可网络交互、Key-Value 对的存储数据库，由 VMware 主持开发，使用 ANSI-C 语言编写，是目前最流行的键值对存储数据库之一。

Redis 是 Remote Dictionary Server 的缩写，我们看看 Redis 官方^[22]给出的具体定义：A persistent key-value database with built-in net interface written in ANSI-C for Posix systems。它是一款开源的、高性能的键-值存储，而不仅仅是简单的 Key-Value 数据模型。Redis 是由 VMware 主持开发，使用 C 语言编写的一种高性能的 Key-Value 类型的开源内存数据库^[23]。Redis 是一个键值对的存储系统，Redis 支持存储多种的数据类型，包含：string(字符串)、list(链表)、set(集合)、zset(有序集合)、hasp(哈希表)。这些数据类型都支持新增/删除、取交集并集和差集以及更丰富的其他操作，而且这些操作要么被完整执行、要么完全回滚的，这种特性叫做原子性^[24]。除此之外，redis 还实现了不同方式的排序功能，为了保证系统性能，所有数据信息都是缓存在内存中，而后 redis 会按照事先设定的频度把存在更新操作的数据写回磁盘，或者在记录文件中追加写入变更操作指令，在这些特性的基础上实现了 master-slave(主从)同步^[25]，redis 还具有很多其他功能，功能示意图如图 3.1 所示。

Redis 是一个性能和评价都相当高的 key-value 数据库。Redis 产品的出现，补偿了非关系数据存在的多种不足，并且在部分场景下能给关系数据库提供补充作用。它提供了 Java, C/C++, C#, PHP, JavaScript, Perl, Object-C, Python, Ruby, Erlang 等多种语言的客户端，对用户来说不用考虑基础语言的限制，使用起来相当方便。

Redis 因为是一个内存数据库，因此在性能方面表现出了优异的性能数据，官方的 bench-mark 的测试数据^[26]：50 个并发进程执行 100000 个请求，操作系统 Linux 2.6,读的速度是 110000 次/s,写的速度是 81000 次/s 。

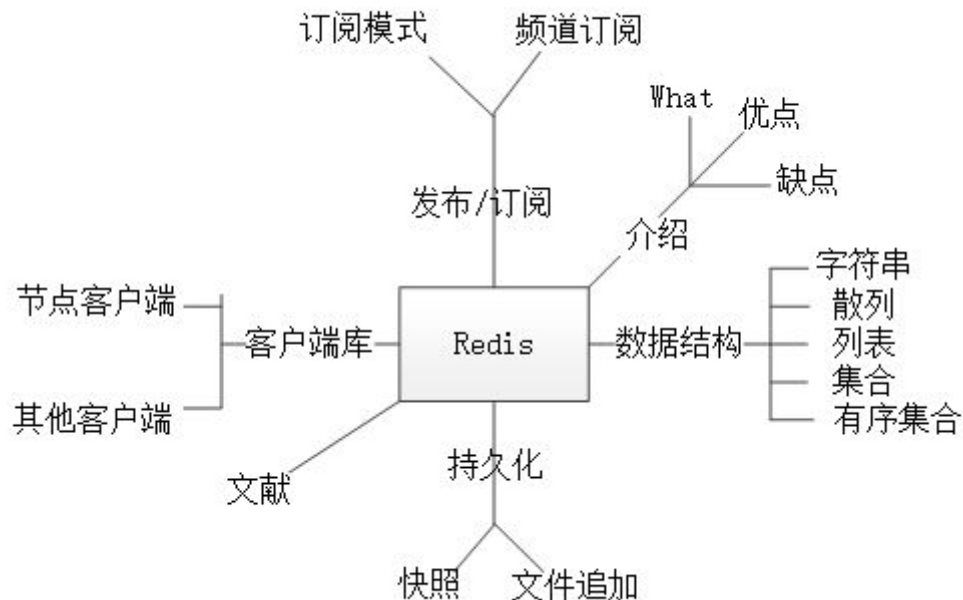


图 3.1 Redis 数据库功能

3.2 Redis 基础特性

根据 Redis 的定义，我们可以看到 redis 具备很多灵活高效的数据结构，同时还具有持久化、主从复制等很多基础特性，下面我们就来全面的了解 redis 的每个基础特性。

3.2.1 Redis 数据结构

Redis 的 Value 支持五种数据类型：String(字符串)，Lists(字符串列表)，Sets(字符串集合)及 Zsets(sorted sets-有序字符串集合)和 Hash (哈希表)。此处需要特殊说明的是：

- (1) Redis 定义的 Key 不能太长，最好不要超过 1024 个字节，不然不但不能提升效率，还会消耗系统内存；
- (2) Key 也不能定义太短，不然可读性降低；

(3) 在同一个项目中定义 Key-Value 最好使用统一的命名规则。

在对几种具体数据结构做详细介绍之前，我们先看看 redis 内部是如何管理这些数据结构的，如图 3.2 所示：

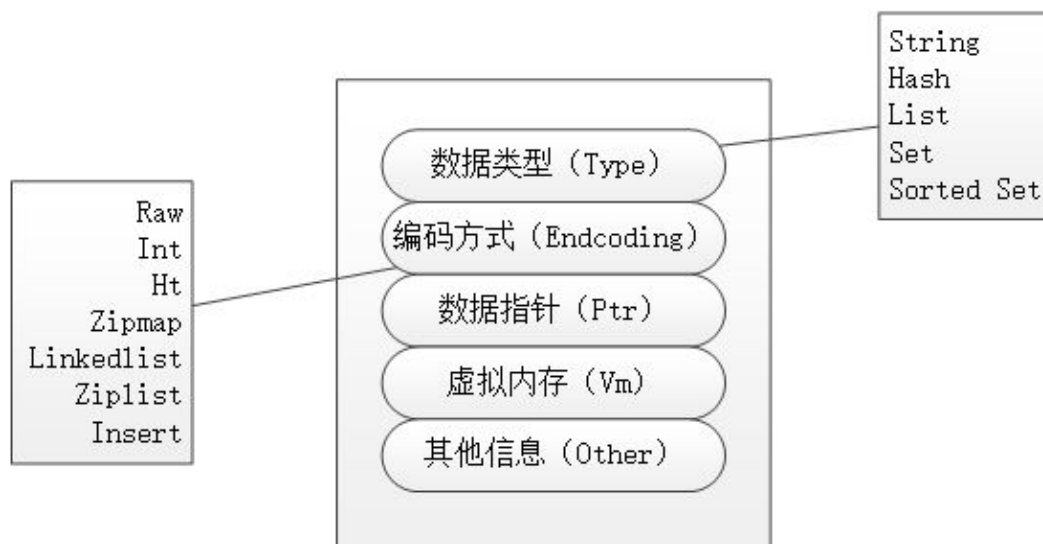


图 3.2 Redis 核心对象

Redis 内部存在一个 `redisObject` 对象来管理所有的 key 和 value^[27]：Type 代表的是 value 对象的数据类型，Encoding 是 `redisObject` 的编码方式，指不同数据类型的内部存储方式，我们通过一个例子来理解一下，例如我们创建一个 String 对象，存储的 value 是字符串，对应编码方式可以是 raw 或者 int，如果是 int 则代表该字符可以用数值来代表和存储这个 String 对象。

下面我们来逐一的了解一下这 5 种数据类型及使用方法：

(1) String

String 是最基础、最常用的一个数据类型。对应的 value 值默认情况下是字符串，但还可以是数值型，因为对 string 执行 `incr`、`decr` 的命令处理时，redis 会自动转换类别为 int 型用于计算。redis 只使用 string 类型就能实现 Memcached 的功能，而且性能更好，除了有和 Memcached 同样的处理命令 `set`、`get`、`incr`、`decr` 之外，redis 还提供了更丰富的字符串操作如表 3.1 所示。

(2) Lists

Lists 是 Redis 另外一个常用且重要的数据结构，从底层设计和实现的角度上说，Lists 并不是数组，而是链表，意思也就是说对于一个 Lists，在头部和

表 3.1 String 操作命令

命令	描述	特性
mset/msetnx	批量设置多个 key 的值，前者覆盖已存在 key 的值后者不覆盖存量 key 值	msetkey1 value1 key2 value2 nx 为 not exists
mget	批量获取多个 key 的值，没有则返回空	mget key1 value1 key2 value2
setex	设置 string 类型 key 值，并设置有效期	setex key1 15(sec) Hello
setrange	重置 key 对应的 value 子串	若重置后的 value 没有以前长，则超出部分被保留
getrange	获取 key 对应的 value 子串	getrangekey1 06
getset	重置 value 值，并返回修改前的值	getset key1 newvalue
incrby/decrby	执行自增/自减特定值	若 key 不存在则设置 key 且 value 默认为 0
append	给特定 key 追加 value 并返回总长	
strlen	获取指定 key 的 value 长度	

尾部分别插入一个新记录所消耗的时间复杂度是常数级的，所以 Lists 本身的数据量大小并不会对时间复杂度造成任何影响。这算是 Lists 的优势，但同时也有劣势，那就是在链表 Lists 元素中定位时会比较慢，而在数组中定位则会快许多。

Lists 常用的操作指令有 Push 和 Lrange，Lpush/Rpush 是用来在列表的头和尾插入新增要素，Lrange 则是在 Lists 指定的一个范围区间内提取需要的记录信息,更多丰富操作命令如表 3.2 所示。Lists 在实际应用中有很强大的功能，例如：利用 lists 实现消息队列排行榜。

(3) Sets

Sets 是一种无序的集合，集合内的元素没有先后的顺序关系。Redis 对于集合的操作也很多，例如：新增/删除元素、取交/并/差集。Sets 操作命令如表 3.3 所示。

表 3.2 Lists 操作命令

命令	描述	特性
lpush/rpush	在 key 对应的 list 头部或尾部添加一个要素	l=left r=right
lrange	显示 list 的要素信息	lrange 0-1//显示全部要素
linsert	在 key 的 list 指定位置插入要素	linsert list before two one
lset	设置 list 指定下标元素的值	key 不存在或越界时都会报错
lrem	从制定 list 中删除 n 个同 value 的要素并返回删除个数	lrem list 1 n hello
lpop/rpop	从指定的 list 头或尾删除一个要素	l=left r=right
lindex	返回 list 某个位置的要素	lindex list 1 2
llen	返回 list 中的要素个数	llen list 1

集合的使用场景也有很多，常见的方式有：大众的社交类软件中都有好友互动的应用场景，例如互相评价印象，互相贴标签，这些印象标签“帅哥”、“暖男”等都可以使用 redis 集合来存储，通过把好友对用户的评价存放在同一个集合中的方式实现。

（4）Zset

Redis 关于集合的操作提供了两种方式：无序集合（sets）和有序集合（sorted sets）。有序集合的每个元素都有一个序号属性，是顺序排列的依据。通常情况下我们将 redis 的有序集合称之为 Zsets，它是 sets 的一个升级版，在集合的基础上，增加了元素的排序功能，有了排序功能每次有新要素插入时集合的部分要素会重新排序，Zsets 的操作命令如表 3.4 所示。

表 3.3 Sets 操作命令

命令	描述	特性
sadd	在集合中增加一个要素	sadd mysets value1
smembers	显示 sets 的全部要素	smembersmysets
srem	删除集合中的一个要素	srem mysets1
spop	随机删除集合中的一个要素	spopmysets
sdiff	获取两个集合的差集	sdiffsets1sets2
sinter	获取两个集合的交集	sintersets1 sets2
sunion	获取两个集合的并集	sunionsets1 sets2
smove	把 a 集合中的指定要素移到 b 集合里	smove a b hello
sdiff/sinter/ sunion-store	对特定集合求交集、差集、并集，并且存储在另一个集合里	sdiff/sinter/ sunionstore sets3 sets1 sets2

(5) Hashs

最后介绍 redis 的最后一个数据类型 Hashs，即哈希。哈希是在 redis-2.0.0 版本后才新增加的数据结构。Hashs 在 redis 内部实际上是一个 HashMap，根据元素的数量有两种不同的实现模式：当元素数量少，为了节省内存，内部会采用类似一维数组的方式来紧凑存储，当数量不断增大时，才真正转换为 HashMap。

它是一个 String 类型的 field 和 value 的映射表，它的添加和删除都是平均的，hash 特别适合用于存储对象，对于将对象存储成字符串而言，hash 会占用更少的内存，并且可以更方便的存取整个对象，它和 java 的 HashMap 完全类似，命令说明如表 3.5 所示。

表 3.4 ZSets 操作命令

命令	描述	特性
zadd	在集合中增加一个要素并排序	zadd sets14 value1
zrange	获取集合中的要素及排序	Zrange sets10-1
zrem	删除集合中的一个要素	zrem sets11
zincrby	增加某个要素的顺序号	zincrbysets12value2
zrank	返回要素在集合中的索引位置	zranksets12
zrevrank	返回集合的逆序排位	zrevranksets1 2
zcount	返回集合给定区间的要素个数	zcountsets1 36

表 3.5 Hashs 操作命令

命令	描述	特性
hset	设置 hash 的 field 为指定值，如果 key 不存在则先创建	hset tab ke1 val1
hget	获取指定 hash 的指定域值	hget tab ke1
hsetnx	设置但不覆盖 field 值	nx 为 notexists
hmset	批量设置 hash 的内容	
hmget	获取 hash 表的全部 key 值	Hmget key field1 field2
hincrby	给 hash 表的某个字段增加值	
hexists	判断 hash 表中某个 key 是否存在	
hlen	返回 hash 表中的 key 数量	
hdel	删除指定 hash 表的某个键值对	
hkeys	返回 hash 表中所有的 key	
hvals	返回 hash 表中所有的 value	
hgetall	获取 hash 表中所有 key 和 value	

3.2.2 Redis 持久化

在了解了 redis 数据结构后,我们发现 redis 确实为我们提供了丰富的内存数据结构类型,我们可以结合实际应用场景背景选择更合适的数据类型来丰富应用的功能和提升应用的性能,但是同时也面临一个难题:怎么才能把这些复杂的内存组织方式持久化到磁盘上?

Redis 为我们提供了四种持久化的方式,分别是:定时快照方式(snapshot)、基于语句追加文件的方式(aof)、虚拟内存(vm)和 Diskstore。这四种方式根据设计思路的不同可以分为两类,前两种方式主要是基于小数据量的磁盘落地功能,即全部数据内存化;后两种方式是基于大数据量的数据存储,不过目前后两种模式仍在实验阶段,并且 vm 模式已经被作者放弃,所以实际的使用过程中,目前最常用的仅前两种,下面我们对这两种模式做分别介绍:

定时快照(snapshot)又称之为 RDB,就是按照事先约定好的时间频度,将增量的 redis 存储数据以快照文件的形式存储在磁盘介质上的方式;AOF 全称是 Appendonly file,直译就是附加文件,实现原理就是 redis 把数据变更指令都追加在文件记录里,当重启 redis 时,读取文件内的所有操作指令并且按照记录顺序执行一遍即可。

(1) RDB 方式,是将 redis 数据库某个时点的数据信息以快照文件的形式保存到磁盘的持久化方法^[28]。

RDB 方式实际是在 Redis 内部存在一个定时器机制,扫描进程按照设置频度去检查数据的当前状况,包含变更次数与时间要素是否满足配置触发持久化的条件,当达到出发条件时操作系统会单独创建(fork)一个子进程来进行数据持久化的操作,子进程默认与父进程享有共同的地址空间,这样子进程就可以遍历整个内存来进行存储操作,主进程此时仍然能正常提供服务,只是不进行 I/O 操作,有写入请求时由操作系统按照内存页(page)为单位来进行 copy-on-write,从而保证主进程的高效性能。子进程数据写入时是将数据先写入一个临时文件中,当整个数据写入完毕后,才会用临时文件覆盖上一个持久化好的快照文件,这样保证了随时系统可以随时的进行数据备份,数据文件总是可用的。

从上面的原理分析,我们可以看出 RDB 模式相对高效,但是缺点也很明显:数据的完整性没有保障。如果应用对数据完成性很敏感的话,就不建议使用这

种模式，因为即时设置 5 分钟备份一次数据，当出现重启时，仍然存在 5 分钟的数据丢失风险。

(2) AOF，即只允许追加变更指令不允许修改已追加内容的记录文件^[29]。

根据上面我们对 AOF 的定义，就是把执行过的写指令按照执行顺序写在记录文件的尾部，重启时读取文件指令明细按照顺序执行一遍就能完成恢复，就是这么一个简单的过程。

配置文件 `redis.conf` 中的 `appendonly` 参数就是控制 AOF 功能的启动与关闭，`yes` 进行开启动作，开启了 AOF 后，如果有写操作，指令就会被追加到记录文件的尾部。AOF 默认的存储策略是每秒钟执行一次，术语称之为 `fsync`，即把缓存中的写指令记录到文件中，这是 `redis` 持久化与性能的最佳平衡点，此策略既能保证 `redis` 有很好的性能表现又能保障数据完整性，最多存在 1 秒钟的数据丢失。在文件追加过程中，如果发生了意外错误，例如系统宕机等意外状况，导致记录文件写入不完整的缺陷，这种情况下，`redis` 提供了一个叫 `redis-check-aof` 的工具可以对不完整的日志文件进行修复处理，这个功能在后续事务处理时，我们也有用到。

AOF 记录文件本身就是增量数据处理的方式，如果不对记录文件做任何处理，则文件一定会随着数据量变动越来越大，有可能导致日志空间不足等意外风险，为了避免这个低级的错误发生，`redis` 提供了一个方便实用的自我保护机制-重写 (`rewrite`) 机制，就是管理员预先设定 AOF 文件大小的阈值，当实际大小超过阈值时，`redis` 就会启动了文件内容压缩，只保留了能保障数据恢复的最小可用的指令集。文件重写功能仍然采用了先创建并写入临时文件，当重写过程结束后，才更名及覆盖上一个可用的 `aof` 文件模式来保障备份文件的随时可用。

AOF 文件本身是可读且可编辑的，我们通过一个设定场景来看看可读可编辑能给我们带来的好处：假设我们在操作 `redis` 时，不小心执行了 `flushall`，内存数据全部清空了，但是开启了 AOF 功能且 AOF 文件没有被重写的前提下，我们可以暂停 `redis` 并对 `aof` 文件进行编辑，删除文件末尾保存的 `flushall` 指令，重启 `redis`，内存数据就能恢复。

从上面的分析来看，AOF 在数据完整性方面有很大优势，也有很多其他优点，不过在同数据规模下，AOF 文件要比 RDB 文件大，恢复速度也远低于 RDB 文件，所以针对数据敏感性不是非常强的应用，还是建议使用 RDB 持久化模式。

3.2.3 Redis 主从复制

Redis 为我们提供了类似 MySQL 一样的主从同步功能，并且支持多种逻辑结构，像一主多从或者多层级从结构都是支持的。主从同步功能主要为了解决数据的冗余备份和提升系统性能的作用。我们先来通过以下方面了解 Redis Replication 的特点和优势：

- (1) 同一个 Master 可以支持多个 Slaves 同步；
- (2) Slave 之间也可以进行连接和数据同步操作，为的是有效缓解 Master 的同步压力；
- (3) Master 与 Slaves 同步是异步进行的，在 Master-Slave 同步期间，客户端仍然可以提交查询或修改请求，不会影响对外的处理性能；
- (4) Slave 之间也是以非阻塞的方式来完成数据同步的处理，同步执行期间，被同步的 slave 对于客户端的查询请求响应都是未同步之前的存量数据。
- (5) 为了缓解和分担 Master 的压力，控制 Master 来完成写操作，设置专用的 Slave 为客户端提供只读服务，大幅提升系统读写效率，系统伸缩性也有很大提升。
- (6) 数据持久化的操作也可以由 Slaves 完成，关闭 Master 的持久化功能，从而保障主服务器的处理性能。

了解了 redis 主从同步的特点和优势后，我们在深入了解一下主从复制的工作原理：从服务器启动并与主服务器成功建立连接后，它会主动发起 SYNC 请求，Master 接收到同步请求时会调用 BGSAVE 指令来创建一个专用子进程来完成数据持久化处理，全量数据持久化处理可以分成两个阶段：首先将主服务器的存量数据都写入 RDB 文件，Master 在数据持久化期间产生的增量数据会启动另一个后台存盘进程，把增量的写指令记录全部缓存在内存中，等待存量数据持久化完成后，Master 把 RDB 数据库文件传送给 Slave，Slave 接收到数据库文件后将其存放在磁盘上并逐步完成数据加载，这样存量数据就同步结束了，然后要进行同步的是持久化期间的增量数据，Master 将所有缓存在内存中的写指令按照约定的 redis 协议格式发给 Slave，Slave 接收后在本地执行这些数据写命令，从而达到最终的数据完全同步。

不论是那一种逻辑结构，Master 只会执行一次持久化动作无论有多少个 slave 有同步请求，然后把持久化好的 RDB 文件分发下去。Redis2.8 的版本提供了数据增量同步策略，此版本之前如果 Master 和 Slave 之间的链接出现断开的

情况下, Slave 会自动重连 Master, 但是在连接成功后, 都是进行全量数据同步。增量数据同步策略是主服务器会在内存中维护一个缓冲区, 缓冲区内是待同步数据, 主从连接成功后, Slave 会把“申请同步的主服务器 ID”和“请求的数据的偏移量 (replication offset)”发送给 Master, Master 收到增量同步请求时, 根据上送的申请同步的主服务器 ID 去匹配自己的 ID 信息, 匹配成功后检查自己的缓存区数据是否能满足申请的数据偏移量, 如果两个条件都满足, 则能完成 master-slave 的增量数据同步。

3.2.4 Redis 事务处理

事务通常具备 ACID 四个属性: 原子性、一致性、隔离性和持久性^[30], 我们来看看 redis 的事务机制是否都具备这四个属性呢?

(1) 原子性

Redis 的命令执行时原子性的, 但是 Redis 的事务处理队列中执行命令有误并不会导致事务的停止, 也不会导致已执行命令的回滚, 这一点就违背了事务的原子性, Redis 的事务可以理解为“将多个命令生成一个批量, 然后一次性、按批量明细的先后顺序执行”的机制, 批量执行期间不会主动中断, 服务器只有在执行完一个客户端的事务后, 才会接受其他客户端的命令请求。

(2) 一致性

Redis 的一致性可以分为入队错误、执行错误、进程被终止三个方面讨论: 首先在命令入队操作时, 如果发送了错误的执行命令, 服务器就会拒绝执行入队操作, 没有入队自然不影响一致性; 其次正常入队后, 命令执行过程发生了错误, 但不会导致回滚也不会影响后续命令执行, 所以对事务一致性不影响; 最后进程被终止, 例如被 KILL 或者发生服务器宕机等事件, 那么如果没有开启持久化, 内存都会被清空, 数据一定是一致的, 如果开启了 RDB 模式, 事务执行过程中, 一定不会执行 RDB 快照的保存工作, 没有 RDB 文件, 恢复的还是事务执行前的数据, 如果开启了 AOF, 但事务没有被写入, 那么恢复自然也没有事务处理结果, 如果事务被写入部分, 那么重启 AOF 时会报错, 必须通过工具对事务进行移除修复才能重启成功, 总结为都不会影响一致性的原则。通过以上分析可以看出 redis 是符合一致性要求的。

(3) 隔离性

Redis 是单进程操作，只有当一个事务结束后，才会接受另外一个事务请求，所以 redis 的事务总是有隔离性的。

(4) 持久性

根据一致性的进程被终止情况的分析来看，Redis 事务处理是不具有持久性的。

从上面的分析来看，由于 redis 不具备原子性和持久性，所以在实际应用时可能存在以下问题：

(1) 事务队列开启后，穿插的查询语句也不会有正确的返回结果，这样如果后续的事务操作需要依附于前一个的查询结果的话，redis 就不能完成这种执行模式。

(2) 事务队列开启后，每条命令入列时返回结果都是 `queued`，那么入队的操作是直接在客户端实现还是与服务端通讯后实现的呢？我们来看一段 redis 的源码：

```
int processCommand(redisClient *c) {
    ...
    /* Exec the command */
    if (c->flags & REDIS_MULTI &&
        c->cmd->proc != execCommand && c->cmd->proc !=
discardCommand &&
        c->cmd->proc != multiCommand && c->cmd->proc !=
watchCommand)
    {
        queueMultiCommand(c);
        addReply(c,shared.queued);
    } else {
        if (server.vm_enabled && server.vm_max_threads > 0 &&
            blockClientOnSwappedKeys(c)) return REDIS_ERR;
        call(c);
    }
    return REDIS_OK;
}
```

从源码中我们发现本身是执行一个批量命令，但是每个命令都需要与服务端进行网络交互，会造成网络流量的额外支出。

为了解决上面的问题，我们在实际应用中考考虑可能的解决方案有：

(1)不同的服务器端来实现读写分离功能;不再使用 redis 本身的事务机制，通过客户端代码实现事务的批量执行，这样就能很方便的的控制回滚和提交，期间穿插查询动作也不影响后续的事务执行了；把查询逻辑放在事务开始之前，这样查询结果就可用了。这三种模式都能很好的解决事务中穿插查询结果不可用的问题。

(2)多使用批量命令，例如 mset/mget；不同的更新操作使用 lua 脚本，一次性发送到服务端。这样都节约了网络资源。

3.2.5 Redis 常用应用场景

Redis 为我们开创了一种新的数据存储思路，我们可以在很多实际场景中，利用 Redis 灵活多变的数据结构和数据操作，设计更合适的应用场景，下面我们挑选几个 redis 比较适合的一些应用场景：

(1) 获取最新 N 条数据记录

例如某游戏平台需要获取 N 个最近登录的用户 ID 列表，我们通过 lists 操作就能完成应用场景的设计

//将登录用户的 ID 及登录时间添加到列表内

```
ret = r.lpush("login:last_login_times",uid)
```

//设定列表仅保留 N 个有效数据

```
ret = redis.ltrim("login:last_login_times",0,N-1)
```

//获取前 N 个登录用户信息

```
last_login_list = r.lrange("login:last_login_times",0,N-1)
```

这种数据结构设计在微博中也很常用，例如获取某个用户的所有博文，或者获取某个微博的最新的 N 条评论，使用 redis 的 lists 很容易就能实现。

(2) 获取 Top N 即排行榜的应用

这个应用场景与第一个应用场景很相像，不过上一个应用以时间为权重，这个场景则以某个条件为权重进行排序，例如微博的评论数排序，提到排序我们就得使用 sorted set 的设计，将权重条件设置为 sorted set 的 score，将对应的值设置为 value，执行 ZADD 命令就能完成数据登记信息。

```
//将微博被评论次数及微博 ID 统一存放在 sortedset 里
zadd comment:comment_times 3000 1
zadd comment:comment_times 2000 2
zadd comment:comment_times 5000 3
//当微博被评论是，被评论次数自增 1
ret = r.zincrby("comment:comment_times",1,id)
//获得被评论次数最多的微博，逆序排列取前 N 名的微博 ID
ret = r.zrevrange("comment:comment_times", 0, N-1)
```

另一种较为普遍的需求场景是按照得分即时排序，这个需求背景下要求实时更新存在变化的数据并对更新后的数据进行排序操作，数据库对于这样的功能则性能表现不够理想。我们看一个典型的在线游戏排行榜的场景：

- 列出综合实力评分前 100 名的用户；
- 查询某用户当前的全服务器实力排名；

这样的场景需求对于 Redis 来讲很简单，即使你的用户量有上百万或者数据变动频率为百万/分钟，这都不成问题：

```
//登记新综合实力评分记录
ZADD leaderboard <score> <userid>
//获取综合实力排名前 100 的排行榜
ZREVRANGE leaderboard 0 99。
//获取某用户的全服务器综合实力排名
ZRANK leaderboard <userid>
```

（3）对要素记录设置失效期限的应用

这个应用场景仍然可以采用 sortedsets，把失效时间设置为 score 值，那么通过对过期时间的排序，我们就可以很简单的定位到过期的数据记录，方便我们定时进行数据清理。

（4）数据去重

数据去重的场景需求使用 Redis 的 set 数据结构最合适不过了，我们在新数据增加时，不用管是否重复，只需要 sadd 到 set 里，set 意为集合，所以会自动排重。

（5）社交类聊天系统

我们知道社交类聊天系统就是要构建实时的消息机制，Redis 的 Pub/Sub 正是完成这样一个功能。

（6）消息队列

消息队列可以理解为带有顺序的消息组成的列表，发布/订阅也是一个消息队列模式，list 和 sorted set 都可以构建消息队列，但后者可以构建有优先级的队列系统，例如：使用 redis 设计资金归集动作，实际上就是一个消息队列，根据列表顺序完成对多个账户的资金扣划，列表的要素扣划完毕，在进行下一步的统一归集动作。

3.2.6 Redis 的注意事项

在了解了 Redis 之后，使用 Redis 开发应用系统会是一件很愉快的事，但是 Redis 像通用的技术一样，存在很多优势的同时，必然也存在劣势和需要注意的内容，因此，在使用 Redis 时，我们一定要注意以下几点：

（1）掌握和控制存储在 Redis 中的键

数据库的主要功能体现在存储数据上，但是对于应用开发人员来说，由于应用程序需求调整或者数据使用策略变更，很容易忽略存储在数据库中的某些数据信息，在 Redis 中同样有这些情况的出现。你可能出于对某个模块的不熟悉或者某个模块被废弃而忘记这些数据用途。

无论是出于什么原因，数据库中肯定会有一些废弃不再使用的数据存在，从而浪费了数据库的空间。Redis 的数据存储模式让集中存储的内容很难被理清，除非你有一套非常成熟的键值命名规则。使用规范的命名规则有助于你对数据库的管理，当通过梳理应用程序或服务，而后进行归类划分，从而按照分类制定一个合理的命名规范时，后期的数据清理、移植和数据使用上就会感觉游刃有余。

Redis 另一个常见用例是作为热点数据的第二数据存储，大部分的非常用数据长久保存在关系数据库中，比如 MySQL 或 Oracle。在这些实例当中，当主数据库的存储数据做清理或移植操作时，开发者会容易忽略二级存储上的关联数据清理。这种跨数据库存储的情况，我们就需要通过关联管理的方法来弥补人主观的遗忘，Redis 有相关的配置特定数据项的所有识别符功能，能保证数据在主存储数据库删除之后，系统根据配置策略，对二级存储的副本信息做相应的清理动作，从而保证数据的一致性。

（2）键名的长度要限制

上文中我们提到过，为了便于数据管理，我们往往要使用规范的命名规则，通过增加前缀或后缀来区分数据流向和用途，这个规则看起来貌似和键名长度限制有相悖的地方。但是，请记住，Redis 是个内存数据库，键名的命名越短需要的空间就越少，对于数据结构简单，键很少的情况，可以忽视这一条，但是对于数百万或者数亿级别的数据量来说，键名的长度就会是影响性能的一个重要因素了。

（3）采用合适的数据结构设计

不管是从内存消耗或者是数据库性能方面来讲，很多情况下数据结构的设计都会产生很重要的影响，我们通过以下一些实例体会一下：对存储数千或者百万的独立字符串的结构，我们考虑是不是可以采用哈希数据结构来取而代之。哈希表上文我们也讲到过，其效率是非常高的，而且可以减少内存消耗，同时哈希表还有利于细节抽象和提高代码的可读性。

不同的模块处理时，可以考虑采用更合适的数据结构，例如：使用 list 代替 set，在不需要使用 set 特性的情况下，List 在更少使用内存的前提下能获得比 set 更高的效率。Zset 是一种成本很高的数据结构，为什么这样讲，因为不管是内存消耗还是基本操作的复杂度，都会比其他数据结构要求高。那么仅仅一个查询语句，无需排序处理的情况下，我认为使用哈希一定比 Zset 要简单。

（4）SCAN 操作时别使用键

Redis v2.8 之后，SCAN 命令已经可以使用，用途是使用游标从 key space 中检索键信息。跟 KEYS 命令相比，SCAN 命令可能不能一次性返回所有匹配到的键信息，但却避免了 KEYS 命令导致系统阻塞的高风险情形，从而让一些检索键的动作可以放在主节点上去完成。

值得注意的是，SCAN 命令基于游标的迭代处理。SCAN 命令每次被调用时，都会返回给用户一个游标，为了完成迭代过程，再次调用 SCAN 命令时，必须以上一个游标作为命令的游标参数，不然无法实现历史数据的延续。SCAN 命令使用时，用户还可以结合 keyname 模式和 count 选项使用。SCAN 命令还可以用在集合、哈希键和有序集合上，具体命令格式分别是 SSCAN、HSCAN 和 ZSCAN。

（5）支持服务器端 Lua 脚本

Lua 脚本的出现，对于 Redis 开发人员来说不得不说是个很棒的消息，不仅给开发人员提供了很友好的开发环境，还大幅度的解放用户的创造力。Lua 脚本使用得当时，会让应用在性能和资源消耗方面得到很大的提升。通过 Lua 脚本无需将数据传给 CPU，就能在最接近数据的地方执行逻辑处理，进而减少网络延时和数据的冗余传输。

在 Redis 使用中，如果遇到数据过滤或数据聚合的应用场景时，我们通过把处理流封装在 Lua 脚本中，使用时调用脚本，消耗最低的资源和使用最短的时间我们就能得到想要的结果集。虽然 Lua 大多数情况下都表现很好，但是也存在弊端：无法跟踪错误信息和错误定位工作。为了弥补这个缺憾，我们可以把 Redis 的 Pub/Sub 和 Lua 结合使用，让 Lua 脚本通过专用通道来推送日志信息，建立一个订阅进程，对日志信息进行跟踪监控。

3.3 性能对比及问题改进

3.3.1 同类产品对比

Redis 属于 NoSQL 数据库范畴，还有很多同类产品，那 redis 与同类产品对比，到底优势体现在哪里？或者对比其他产品来说有哪些短板呢？先来简单的认识一些同类产品。

(1) CouchDB 是一个开源的面向文档的数据库管理系统^[33]，初期基于 C/C++ 语言编写，后期转至 Erlang OTP 平台，支持 REST API 访问，支持双向复制、主主复制，使用 MVCC 模式，因此写操作不影响读，因为具有以上特点，其具有高可用性和高可靠性，而且很容易学习和使用和很强的数据库一致性特性。

(2) MongoDB 是一种面向集合的，基于分布式文件存储，模式自由的文档数据库^[34]。基于 C++ 语言开发，MongoDB 兼具关系数据库和非关系数据库的特征，是非关系数据库产品当中功能更丰富的。他支持松散的数据结构，比如 bson 格式，因此可以存储复杂的数据类型。Mongo 最鲜明的特点是拥有丰富的查询语法，几乎可以实现关系数据库所有的查询操作，而且支持索引的建立。

(3) Cassandra 是一种开源的分布式 NoSQL 数据库^[35]，基于 Java 语言开发，是一个类似 BigTable，功能比 Dynamo 更丰富，但数据类型支撑都不如 MongoDB 的混合型非关系数据库，主要特点是基于集群式部署，具有很高的可伸缩性。

表 3.6 NoSQL 产品参数对照表

分类 指标	Redis	CouchDB	MongoDB	Cassandra
实现语言	C/C++	Erlang	C++	Java
协议支撑	类 Telnet	HTTP/REST	自定义二进制， BSON	自定义二进制 thrift
性能	依赖内存， TPS 非常高	读的性能很低	依赖内存， TPS 较高	写速度高于读
数据类型	数据类型丰富	文档	文档	列族数据模式
内存及存储	虚拟内存，数据具备时效性	存储半结构化数据	镜像文件，适合大数据量存储	
可用性	主从同步，依赖客户端完成读写分离	主主同步，易于多站点部署	主从同步，自动共享，对客户端屏蔽故障转移	可权衡调整的分布式和复制
可靠性	RDB 和 AOF 两种持久化模式	冲突检查功能，注重版本控制	支持 binlog 方式，支持持久化	基于磁盘的持久化
一致性	支持事务	不支持事务	不支持事务	不支持事务
数据分析	不支持	不支持	内置数据分析	不支持
应用场景	较小数据量，全内存存储	静态查询，对版本控制有要求	海量数据访问，需要动态查询	对写的要求高于读

通过上面我们对同类的几个 NoSQL 产品做概念性的介绍，我们发现其实 NoSQL 数据库的大致功能都差不多，只是在细节设计和性能偏重上有所差异，那么差异在什么地方，我们对通过一个表格来进行对比分析，分析数据如表 3.6 所示：

通过 NoSQL 产品参数对照表，我们可以看出产品设计上都很关注数据高并发的读写性能和海量数据存储，与传统的关系数据库相比，他们在数据架构和数据模型方面都做了“减法”，而在扩展和高并发方面做了“加法”。虽然各类产品在设计目标上都很一致，不过也各有特色，正式因为特色的不同，为更多的 Web 应用提供了更加灵活和无限的发展空间。Redis 在同类产品中更是表现出了更多的突出特点和优势：

- (1) 免费开源;
- (2) 易安装使用、操作简单;
- (3) 数据结构丰富灵活, 适用场景广泛;
- (4) 逻辑简单且纯内存操作, 具有 10 万+ 读写性能表现;
- (5) 具有事务性, 保证数据一致性。

不过 Redis 也有它的短板所在:

- (1) 持久化不成熟, 快照文件必然面临数据丢失, AOF 很影响性能;
- (2) 存储成本高, 纯内存操作, 不适合海量数据;
- (3) 注重内存容量规划;
- (4) 架构扩展不是很简单。

3.3.2 问题改进

根据上文中 Redis 与同类产品通过性能参数对比、分析及总结, 可以看出 Redis 在内存容量规划方面和数据复制上还存在问题, 那么针对这两个问题, 我们如何处理才能预防内存崩溃等风险事件的发生呢?

(1) 内存容量规划

先来看一个 Redis 的系统表现: Redis 在物理内存上使用比较多, 但还没有超过服务器的实际物理内存总容量时, 服务器就已经出现不稳定甚至崩溃的现象。曾经以为这种风险是由于同一个物理机上的多个 Redis 实例同时触发持久化的 BGSAVE 而导致瞬间 Fork 出多个 redis 进程, 造成内存占用加倍而导致的。后来经过研究发现这种不稳地现象是由于 Redis 的持久化使用了 Buffer IO 造成的, 即 Redis 持久化文件操作都是基于物理内存的 Page Cache, 而大多数数据库系统往往为了维护一个数据的 Cache 会使用 Direct IO 来绕过这层 Page Cache, 伴随数据量的增大, Redis 持久化文件会越来越大, 尤其是快照文件会比 AOF 文件大数倍, 在对此文件进行读写操作时, 磁盘文件中的数据都会被加载到物理内存中作为操作系统对该文件的一层 Cache, 而这层 Cache 的数据与 Redis 内存中管理的数据实际是重复存储的, 虽然内核在物理内存紧张时会做 Page Cache 的剔除工作, 但内核很可能认为某块 Page Cache 更重要, 而让你的进程开始 Swap, 这时你的系统就会开始出现不稳定或者崩溃了。

为了避免出现内存不足的风险问题, 内存规划管理工作很重要, 我们从前期规划和后期管理两方面着手解决:

我们前面分析过 Redis 的适用场景是全部数据存储在内存在中，而内存容量有限，前期规划就包含对存量的规划和增量的规划，存量相对容易，例如目前存量数据 500G，假设单台服务器物理内存 100G，则大约需要 9-10 台设备来存储。增量规划主要依附于未来业务发展速度，也就是数据增长速率，这种预测性的规划，有一定参考性，但是并不一定完全满足，因为业务数据不一定按照常规速度增长，那么在未来基线无法准确预测的情况下，如何进行 Redis 的在线扩容呢？

Redis 给出了一种叫做 **Presharding** 的方案来解决动态扩容和数据分区的问题，就是在同一台机器上部署多个 Redis 实例的方式，当出现容量不足的情况时，就把多个实例拆分到不同的物理机器上，完成实际的扩容效果。

Redis 实例拆分处理步骤如下：

- 1、在新机器上启动好对应端口的 Redis 实例。
- 2、配置新端口为待迁移端口的从库。
- 3、待复制完成，与主库完成同步后，切换所有客户端配置到新的从库的端口。
- 4、配置从库为主库。
- 5、移除老的端口实例。
- 6、重复上述过程迁移好所有的端口到指定服务器上。

以上拆分过程是一个平滑迁移的过程，不过拆分还是要依赖 Redis 本身的主从复制特性，也就是说在伴随主库快照文件增大时，同步消耗的时间也会加长，同时由于压力会影响主库的性能。所以拆分工作最好选择业务低峰期完成，避免对正常运营的影响。

后期管理可以认为是做好垃圾数据清理工作，可以通过 Redis 自带的数据失效机制，对每个 Key 设定有效期，通过失效期完成数据的定时清理，不过自动过期就需要开发人员在数据创建时就规划好数据的生命周期，不然失效期设置不合理，会导致存在大量空余内存时，Key 过期仍在释放内存，导致缓存命中率的降低，或者就是内存已经不足了，但 Key 还未过期的风险，所以仅靠自动失效机制也不是很靠谱，为了更精准的控制数据有效期，可以结合 ZSET 来解决，利用 ZSET 来维护数据更新频度，使用变更数据戳作为序列的 score 值，每次做更新操作时同步更新时间戳，这样数据有效期就是一个按照更新时间排列的序列串，通过对 ZSET 操作，很容易得到找到最近未使用数据集，然后对这些数据

进行清理，达到精确释放内存的目的。

(2) 主从复制缺陷

上一章节中我们提到了 Redis 的主从复制是主从服务器间的数据冗余备份，主要为了提升系统可用性，防止单点故障问题，我们先来回顾一下复制流程，如图 3.3 所示：

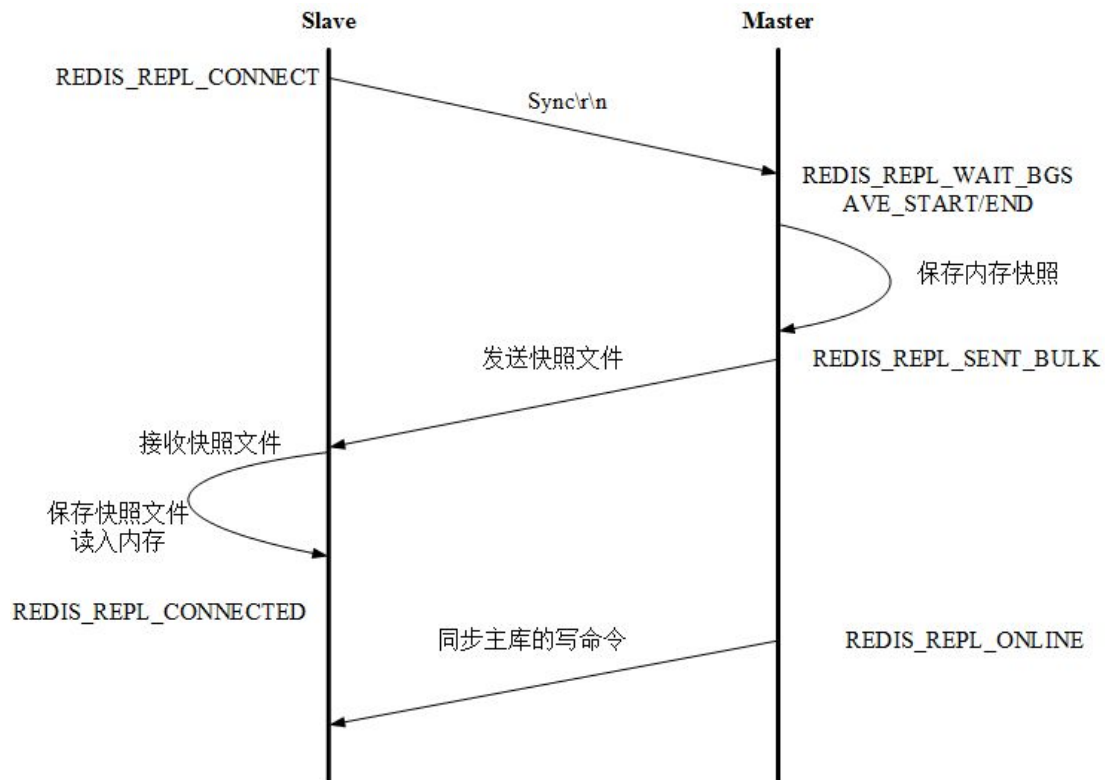


图 3.3 复制流程图

从复制流程图我们可以看出 Slave 向 Master 发出数据同步申请后，Master 都是发送全量快照文件，无法完成增量数据传输，这会给集群架构设计带来困扰。由于是全量数据同步，同步和恢复时间都会随着数据量的增大变的越来越慢，因此如果 Redis 集群配置需要主从复制功能，那么就要规划好所有的从库设置，避免使用过程中的从库扩展。

目前面临的最大的问题是无论是持久化还是主从复制都没有办法解决 Redis 单点问题，即一台 Redis 挂掉了，不能很快的完成数据恢复，通常几十 G 的持久化数据，Redis 重启加载需要几个小时的时间，而复制又有缺陷，如何解决呢？

1、主动复制避开 Redis 复制缺陷

既然 Redis 的复制功能有缺陷，那么我们不妨放弃 Redis 本身提供的复制功能，我们可以采用主动复制的方式来搭建我们的集群环境。

所谓主动复制是指由业务端或者通过代理中间件对 Redis 存储的数据进行双写或多写，通过数据的多份存储来达到与复制相同的目的。主动复制虽然解决了被动复制的延迟问题，但也带来了新的问题，就是数据的一致性问题，数据写 2 次或多次，如何保证多份数据的一致性呢？如果你的应用对数据一致性要求不高，允许最终一致性的话，那么通常简单的解决方案是可以通过时间戳或者 vector clock 等方式，让客户端同时取到多份数据并进行校验，如果你的应用对数据一致性要求非常高，那么就需要引入一些复杂的一致性算法比如 Paxos 来保证数据的一致性，但是写入性能也会相应下降很多。

通过主动复制，数据多份存储我们也就不再担心 Redis 单点故障的问题了，如果一组 Redis 集群挂掉，我们可以让业务快速切换到另一组 Redis 上，降低业务风险。

（2）Redis 复制的改进思路

上面分析 Redis 复制的主要缺陷在于没有增量复制，导致无法快速恢复解决单点问题，那么对持久化方案做改进，是否能完成增量同步？

改进的持久化方案是首先写 Redis 的 AOF 文件，并对这个 AOF 文件按文件大小进行自动分割滚动，同时关闭 Redis 的 Rewrite 命令，然后会在业务低峰时间进行内存快照存储，并把当前的 AOF 文件位置一起写入到快照文件中，这样我们可以使快照文件与 AOF 文件的位置保持一致性，这样我们得到了系统某一时刻的内存快照，并且同时也能知道这一时刻对应的 AOF 文件的位置，那么当从库发送同步命令时，我们首先会把快照文件发送给从库，然后从库会取出该快照文件中存储的 AOF 文件位置，并将该位置发给主库，主库会随后发送该位置之后的所有命令，以后的复制就都是这个位置之后的增量信息了，整体改进思路如图 3.4 所示：

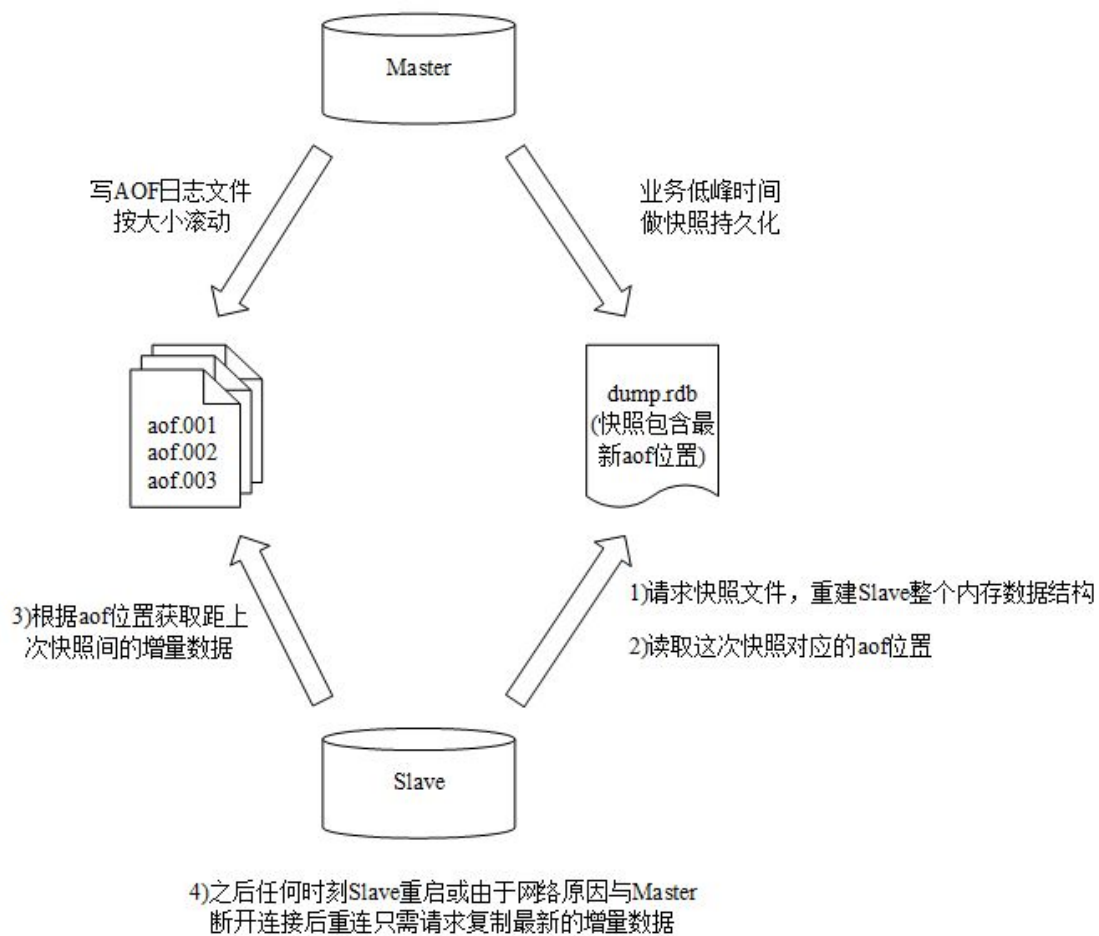


图 3.4 复制改进思路

从与同类产品性能对比后，发现了 Redis 的优势和不足，结合第二章节对 Redis 基础特性的研究和工作原理的分析，针对 Redis 存在的容量规划管理问题和主从复制问题，提出了不同程度的改进方法和手段，以提升 Redis 在未来应用中的服务能力和加深对 Redis 产品的进一步认识。

3.4 本章小结

本章为论文的主题部分，主要介绍了 Redis 缓存技术研究，先通过对 Redis 的定义和特性做了概括性描述，而后重点剖析了 redis 的基础特性，包含数据结构、持久化、主从复制、事务处理等技术难点进行了剖析和讲解，同时对 redis 一些常用应用场景的设计做了介绍，对 Redis 使用时的注意事项进行了列举。而后为了体现 Redis 的特点，我们又通过 Redis 同类产品性能参数对比，进一步确

定了 Redis 缓存技术的优势和不足，为了提升 Redis 缓存技术的性能，我们针对分析出的 Redis 容量规划管理问题和主从复制问题，提出了改进思路 and 想法，这样更进一步加深了对 Redis 缓存技术的认识和理解，同时为 Redis 在未来应用中更好的发挥性能打下了坚实的基础。

4 Redis 在互联网金融服务平台的应用

互联网金融浪潮的迅猛发展，推动着金融领域的创新和改革，本土银行也纷纷在电商领域发力，加快在该领域的创新和尝试。6月25日，某银行依托其电商平台-兴融易再出新招，携手基金公司针对互联网用户度身打造的互联网理财产品-月添利和兴融易手机 APP，率先抢占了中原地区互联网金融理财和移动互联发展的领先地位。同时，与公司合作的投融资产品也将在兴融易电商的 PC 端上线，其互联网金融的产品和服务在持续的提升和完善^[36]。

兴融易互联网金融服务平台的推出，为本土银行在互联网+的改革创立了标杆，标志着该银行在互联网+领域的改革和创新，为该银行在互联网领域的长足发展打下了坚实的基础，是该银行发展过程中的重要里程碑。

4.1 系统概述

互联网金融服务平台包含直营银行和金融电商两大板块，现已形成四大基本功能：投资理财、网络融资、金融电商、生活缴费，推出日添利、月添利等特色理财产品和通用缴费、支付网关等亮点功能。兴融易以普惠金融为设计理念，致力于打造理财更安心，融资更轻松，生活缴费更快捷，电商入驻更简单的服务模式。兴融易将以创新的产品和完善的服务，满足客户不断变化的需求，持续提升客户体验。

相比传统银行业务，互联网金融服务平台具有时效要求高、响应速度快等特点。随着注册用户数量增多、业务品种增加、数据更加复杂，对平台在实时性、响应速度、安全方案、容量管理等方面提出更高的要求。

4.1.1 平台总体架构

互联网金融服务平台整体技术框架采用了 SOA（面向服务的体系结构）架构，完全松散耦合式设计，通过服务治理架构实现服务的拓展和灵活的网络化部署，通过对应用模块的提取和自由组合，充分整合可用的资源，适应业务的快速变化，提高企业竞争力^[37]。平台是基于 Spring3.0 框架^[38]，采用分层架构设计：分布式数据库系统、分布式应用服务器集群、分布式跨平台网络通讯中间

件、分布式文件服务系统等，平台架构具有开放性、灵活度高、轻量化、高性能、松耦合、高可靠、分布式、可扩展等众多优点。

互联网金融服务平台是行内与外部系统交互的桥梁，外联网络连通各种支付渠道、商户服务器、第三方合作伙伴，内联行内系统完成系统内客户信息检查及控制以及内控管理。

我们通过业务总体架构图和总体逻辑架构图来从业务和技术框架两个方面来全面的来认识一下互联网金融服务平台。

先来看看业务总体架构图，如图 4.1 所示：

互联网金融服务平台从业务角度可以分为 4 大功能模块：行内应用、银企平台、电子核心系统、银银平台。

行内应用包含互联网金融服务平台的前端产品：电商服务 WEB、互动银行 APP 和门户网站等；

银企平台是整个互联网金融平台的接口层，负责第三方系统的交互功能，包含：基金公司、企业商户、融资公司、电力公司等；

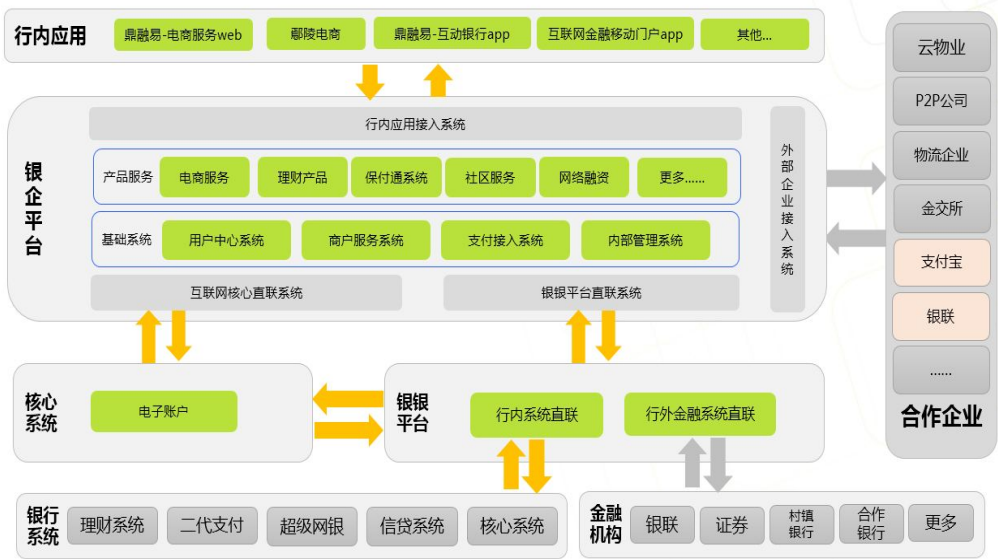


图 4.1 业务总体架构图

电子核心系统是互联网金融服务平台的账户管理系统，负责电商平台客户注册和客户电子账户开立以及电子账户的核算记账；

银银平台是互联网金融平台的支付接口，负责金融服务平台与外围系统的支付结算。

从业务总体架构图我们可以看出互联网金融服务平台具有的所有业务功能，每个业务模块既相对独立又紧密相连，我们在来看看平台的技术逻辑架构图，如图 4.2 所示：

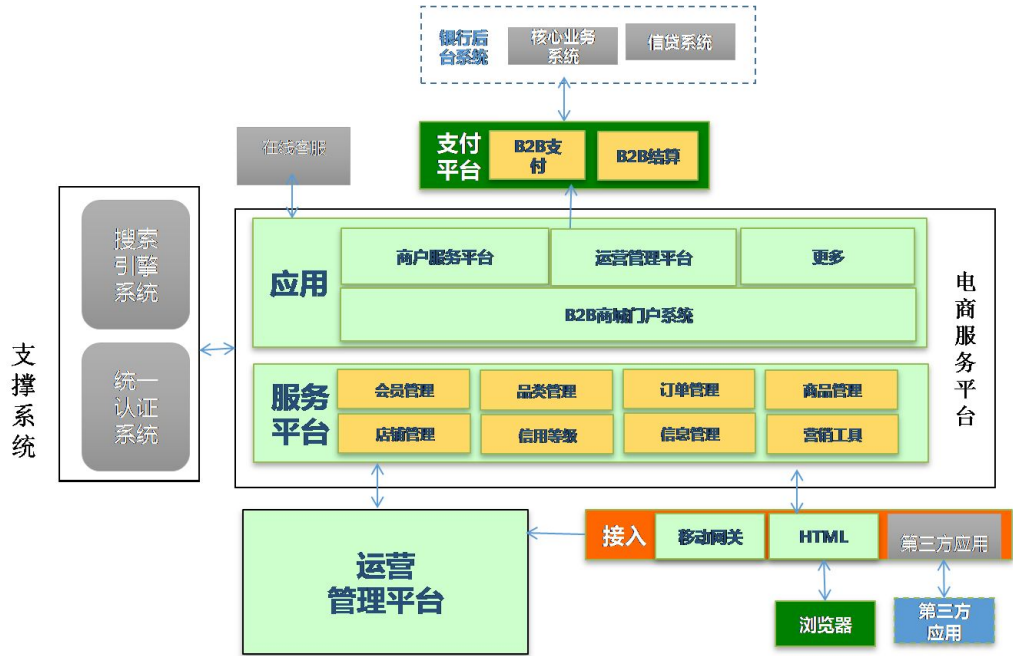


图 4.2 技术逻辑架构图

- 电商平台解决方案由五大部分构成。分别是：
- 1、统一电商服务平台：整个平台的核心系统，提供最基础的电商服务和电子商务应用的管理。其他系统都将与该系统对接，实现对服务的闭环。
 - 2、支付平台：银行的支付平台解决方案，提供 B2B 和 B2C 支付和结算服务。在本期项目主要实现 B2B 支付和结算服务功能。支付平台与支付网关最大的区别在于：支付平台与支付网关对接，支付网关主要负责支付交易的实现，支付平台完成对支付服务和结算服务的封装，并最终支付产品形式提供给用户和企业使用。
 - 3、电商运营管理平台：电商运营管理平台是提供给银行管理人员，如客户经理和电商服务团队使用的运营管理系统。电商运营管理平台是一个独立的应用，不对外部用户开放，只提供给注册并授权的用户使用。
 - 4、支撑系统：为实现平台用户的统一管理和支持客户在平台的关键内容搜索，特定制了统一认证系统和搜索引擎系统。

5、数据系统：面向未来的大数据系统，在前期建设中，主要以数据采集和数据整理和简单数据分析和应用为主。

平台架构采用松耦合方式，并且客户端基于互联网，采用 B/S 架构，可以采用 SSH（Spring+Spring MVC+Hibernate）架构^[39]来保证客户端的灵活性，如图 4.3 所示：

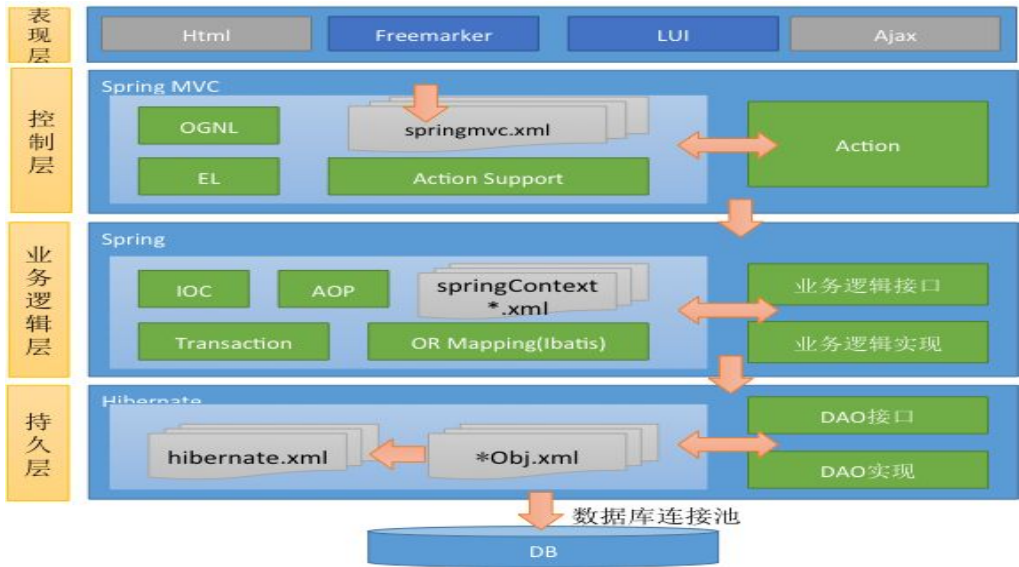


图 4.3 客户端结构图

4.1.2 平台功能介绍

互联网金融服务平台目前已经具备 5 大功能模块：生活缴费、投资理财、网络融资、电子账户管理、电商服务。各功能模块各具特色，不仅在客户社区生活方面提供足不出户的便利生活缴费功能，而且为客户提供投资理财、网络融资的渠道，更为中原地区的企业商户和个人客户架起了网络销售与购物的桥梁。

系统内通用功能流程包含：客户注册、实名认证、开立电子账户、电子账户充值、生成订单、订单支付、订单退款、商户对账环节。通用流程如图 4.4 所示：

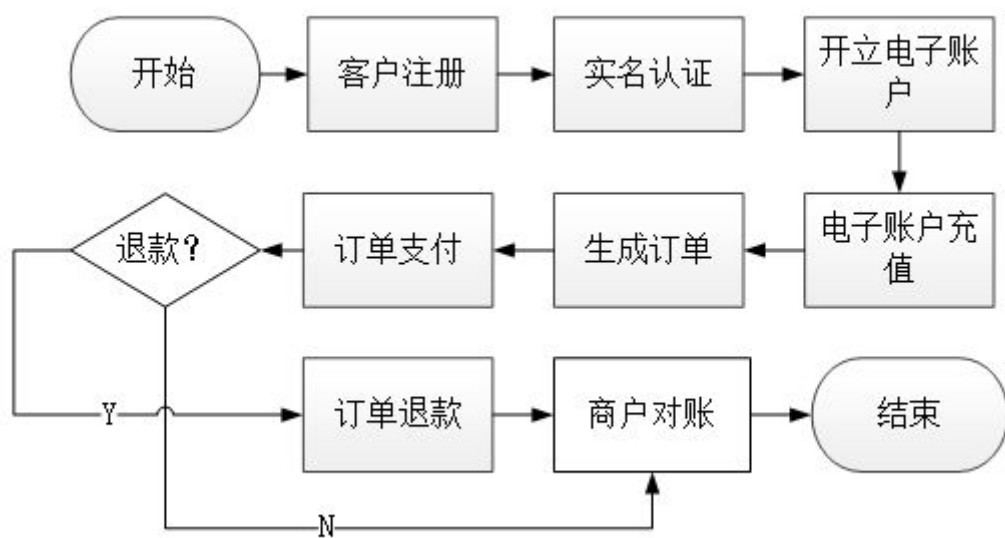


图 4.4 通用流程图

通用流程的客户注册、开立电子账户、充值、生成订单、订单支付、订单退款、商户对账这些环节都是通过应用直接与数据库服务器交互，交互模式如图 4.5 所示：

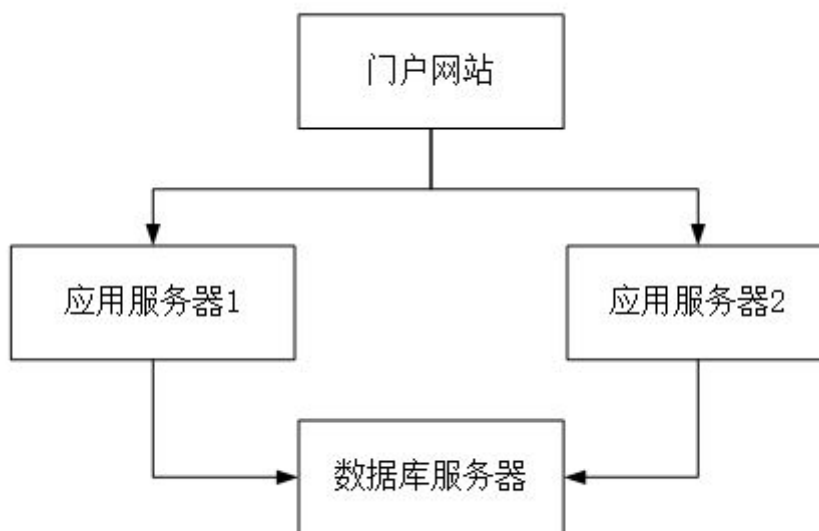


图 4.5 交互模式图

4.1.3 重点流程剖析

在对系统总体架构和功能结构说明之后，需要重点剖析三个业务流程。

（1）客户注册流程：客户通过互联网金融服务平台进行平台客户注册，注册界面如图 4.6 所示：

1 注册平台用户

2 优选客户经理

3 绑定银行卡

注册平台用户

已有账户，[立即登录](#)

* 手机号码：

+86

* 图形验证码：

请输入图形验证码



* 手机验证码：

请输入验证码

发送手机验证码

* 邮箱：

请输入常用邮箱

* 登录密码：

[点此下载控件](#) 6~20位字符，不能为纯数字

* 确认密码：

[点此下载控件](#)

好友推荐：

请输入推荐好友手机号

☐ 同意《[郑州银行电商服务平台注册条款](#)》

同意条款并注册

图 4.6 注册界面

客户需要录入信息包含手机号码、图形验证码、手机验证码、邮箱、设置登录密码并阅读电商服务平台注册条款信息才能完成注册。注册过程中需要注意的是客户发送手机验证码，平台应用会生成一个随机的验证码信息，同时将手机号、验证码、发送时间戳和验证状态信息登记在 Oracle 数据库中，出于安全考虑，该验证码有一个有效期为 5 分钟，这个背景下就要求系统存在一个后台进程扫描该验证码的时间戳与当前系统时间做比对，当发现该手机号的验证码未验证且发送时间戳已经大于 5 分钟，则数据库需要删除该数据信息，支持客户再次发起手机验证码申请，当然在 5 分钟之内由于平台与运营商问题造成

的验证码收不到情况，客户可以通过发送验证码按钮访问数据库重新获取数据库已登记的验证码信息，平台处理流程如图 4.7 所示：

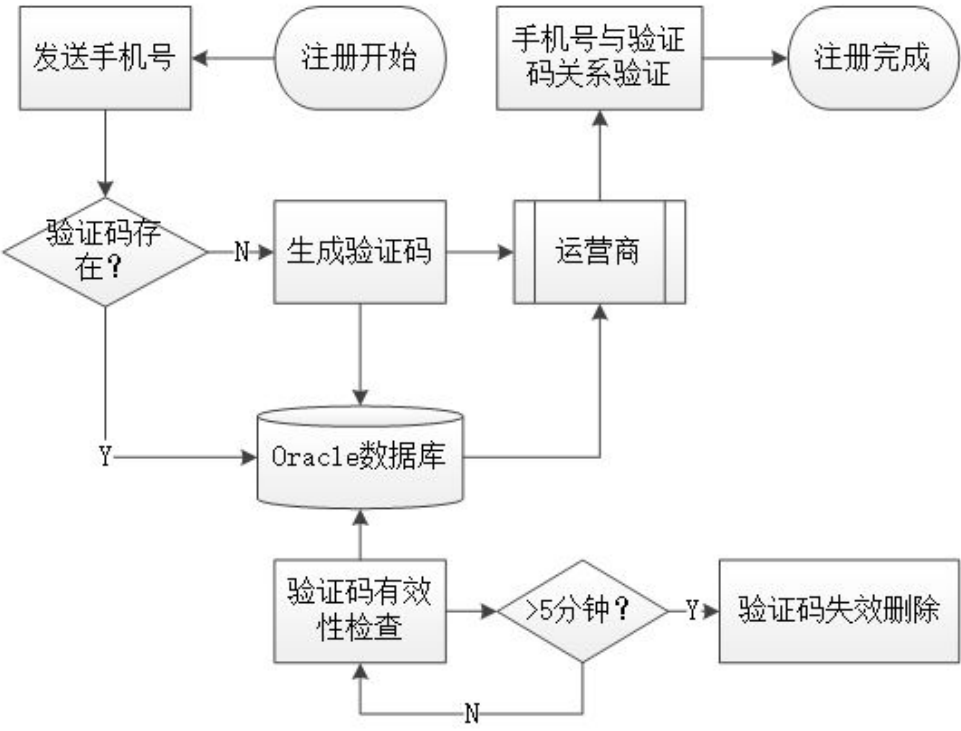


图 4.7 客户注册流程图



图 4.8 类目展示图

(2) 电商门户类目处理流程：电商门户网站首页有关于商品类目的信息展示，为客户浏览和查询相关商品提供方便和支持，界面展示如图 4.8 所示：

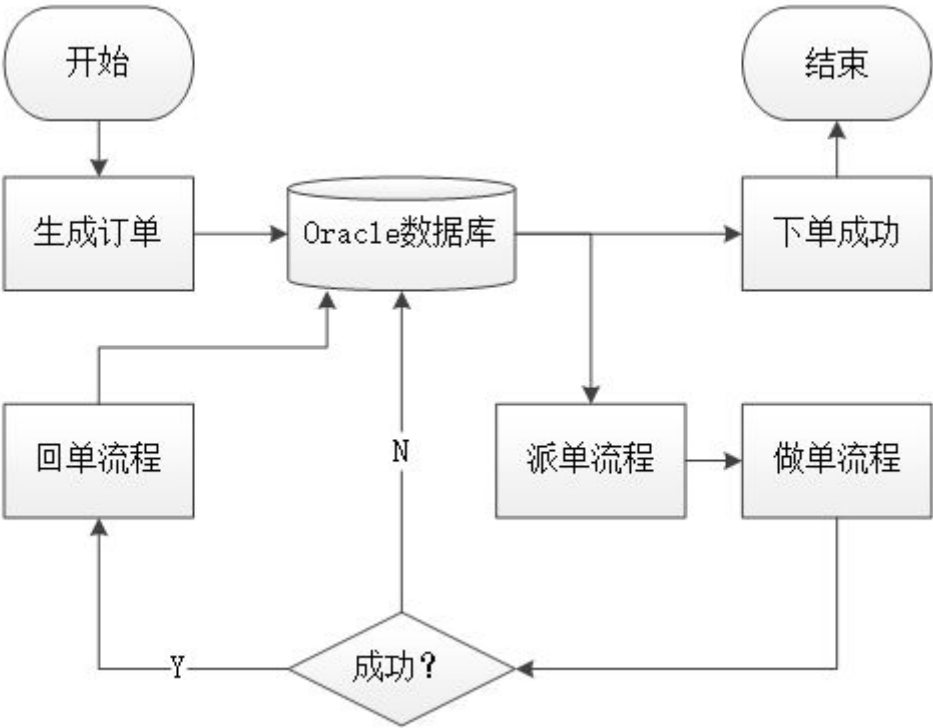


图 4.9 商城订单流程图

电商平台的商品类目的制定和更新是由银行的电商平台管理人员通过后台管控系统来完成具体的操作和实现，例如：新增商品类目、调整商品类目明细等等操作，后台管控系统是直接跟 Oracle 数据库通讯，进行相应的 insert、update、delete 等操作，当客户登录电商首页时，Web 请求 Oracle 数据库返回当前的产品类目信息，Web 应用收到数据库返回后进行界面拼装操作。

(3) 商城订单流程：客户通过电商门户，发起涉及多个商户的整体订单，系统将订单信息登记在数据库的订单信息表，应用有专用进程扫描订单状态，对已下单的订单进行派单操作，同时修改订单状态为已派单，订单派单之后，会有相应应用完成做单流程，做单流程及发起与商户应用的交互，确认商户是否能满足客户的该订单信息，若商户能满足，则做单成功，调用回单流程给数据库修改订单状态为下单成功，同时返回客户端下单成功消息，在做单过程中，若该笔订单涉及商户为 1 至 2 个，则一次就能完成做单流程，若订单设计商户

数目较多需要跟多家商户轮询通讯，而且通讯过程有可能由于网络等问题造成与商户的通讯超时，直接导致一次做单无法完成的情况，因此一次做单不成功的订单，又会返回数据库进行重新派单操作，循环处理直至做单成功。下单流程如图 4.9 所示。

4.1.4 系统瓶颈

上一个章节里我们通过对系统内当前几个重点流程的剖析可以发现，目前平台的相关业务流程都依附于对 Oracle 数据库的访问，高并发和大数据量的访问会给 Oracle 数据库带来非常大压力，而目前数据库采用的是 HA 的灾备模式，应用采用双活的负载均衡模式，显然应用端没有很大的访问压力，但是对于数据库服务器的压力则是巨大的，当数据库服务器不堪重负宕机，则直接导致平台所有业务的终止，乐观点看，数据库服务器配置高，幸免于宕机，但是每个流程都与 Oracle 数据库服务器通讯，并且是并发量很高的情况下，直接带来的客户体验是网站很慢，没有响应。

我们已经发现系统的关键平键问题在于关系数据库，那么关系数据库为什么在当前应用架构设计中显得不那么给力呢？我们来试着分析看看：

关系数据库是以严格的关系模型为基础^[40]，通过关系表示事物之间的联系，并且以数据表的形式展现，数据表结构都是事先定义好的，也就是固定的且不易更改和扩展的，这样在面对 Web 应用灵活多变的交易模式和交易场景时，关系数据库的局限性就自然暴露出来了。同时目前金融服务平台需求大数据量存储和高性能并发，每秒钟可能有上万次的读写请求，而关系数据库最优模式下也就能勉强支撑上万次的查询，但上万次的写操作，Oracle 是没有办法做到的，由于请求的增多，数据量也会向亿级发展，Oracle 如果同一张表拥有几十个字段并且有亿级的数据量，那 SQL 的执行效率会低的无法忍受的。并且对于 Oracle 数据库支撑的系统其数据库可扩展性和高可用性能都不高，由于受到关系数据库同步等技术的限制，无法对关系数据库设置集群模式，这种情况下，如果需要对数据库性能做提升，就必须停机升级数据库配置，影响平台 7*24 小时对外服务的要求。

通过我们对关系数据库局限性的分析，我们可以清楚看到 Oracle 数据库对互联网金融服务平台的支撑在数据库高并发 I/O、高效存储和海量数据、高可扩展性和高可用性等方面都不适合，而 Web 应用对比传统应用最大的特点也在于

这些方面，如果无法满足以上几个方面的需求，那么可以说我们的互联网金融服务平台是失败的。

因此，我们可以确定目前互联网金融服务平台的系统瓶颈就在于数据库设计的不得当。

4.2 Redis 在缓存架构设计中的应用

通过上一个章节我们对当前系统架构和瓶颈问题分析，我们可以发现 Oracle 数据库在这种实时性要求非常高的互联网应用领域，它的缺点就会被放大，变的显而易见。为满足互联网金融服务平台的正常运转，达到客户满意度较高的一个实时性交易平台效果，业务管理部门对系统处理的性能提出更高的要求，传统的系统架构模式已经不能满足当前的业务发展要求。

对于这种新兴的互联网应用，因其自身具备实时性要求高、数据量大、并发访问高和数据类型多而杂等特点，所以仅使用关系数据库是无法满足系统架构要求的，那么结合内存数据库速度快、扩展性好的特点，设置内存数据库在关系数据库前端来完成数据缓存处理是个不错的架构设计方案。内存数据库的数据操作都是直接存放在内存里，相比磁盘操作的关系数据库来说，性能高出很多，这一点正是互联网金融平台所急需的。第 3 章中我们详细的介绍和研究了 Redis 缓存技术，得出了 Redis 兼具内存数据库和非关系数据库 Key-value 数据模型的特点，通过同类产品对比，其拥有自身特色。结合 Redis 缓存技术研究和系统架构分析，我们得出这样的结论：Redis 的各种特点都很适用于互联网金融服务平台。

4.2.1 系统缓存架构设计

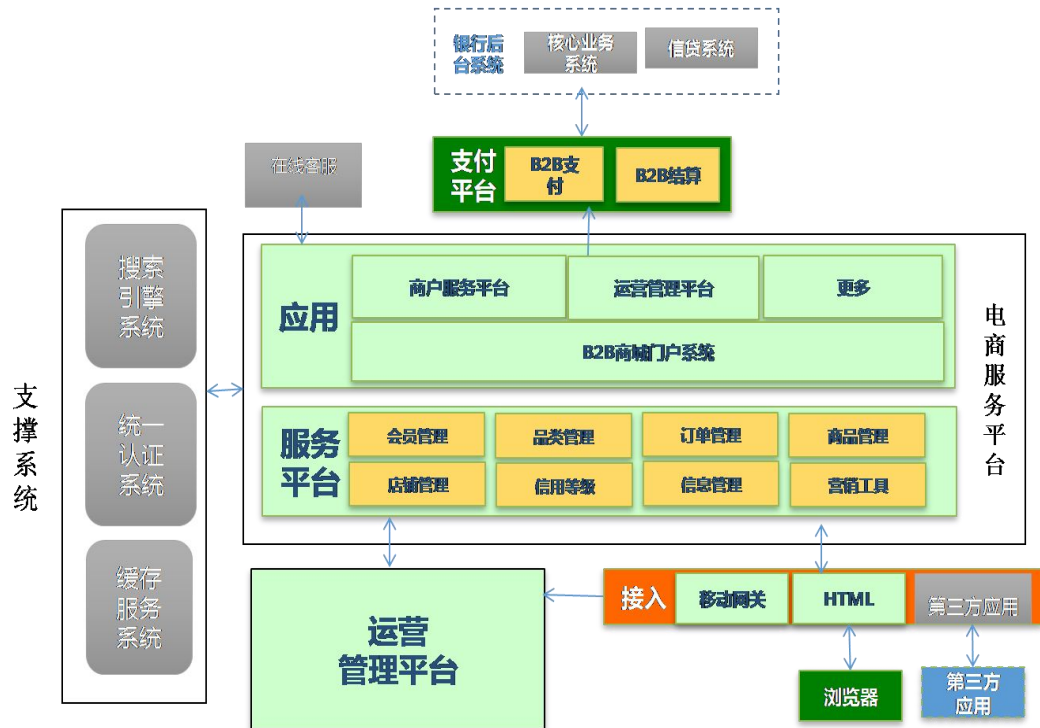


图 4.10 缓存设计的逻辑架构图

针对系统架构没有缓存模块设计，导致应用直接与 Oracle 数据库服务器交互导致的响应慢、超时率高的问题，我们对系统架构通过在支撑系统内增加缓存服务器的手段，把一些不需要直接访问数据库或者对数据库压力较大的流程指向到缓存服务器端，由缓存服务器完成相应处理，来减轻数据库压力，提供系统性能，改造后的系统逻辑机构如图 4.10 所示：

平台整体架构我们已经加入了缓存服务器的支撑，我们在把目光聚集到缓存服务器的架构中，因为我们是把 Redis 内存数据库作为 Oracle 关系数据库的前端补充来做，他们自然不是相互独立存在的，我们需要先用 Redis 技术完成缓存处理，在持久化到 Oracle 数据库中，缓存服务器架构如图 4.11 所示：

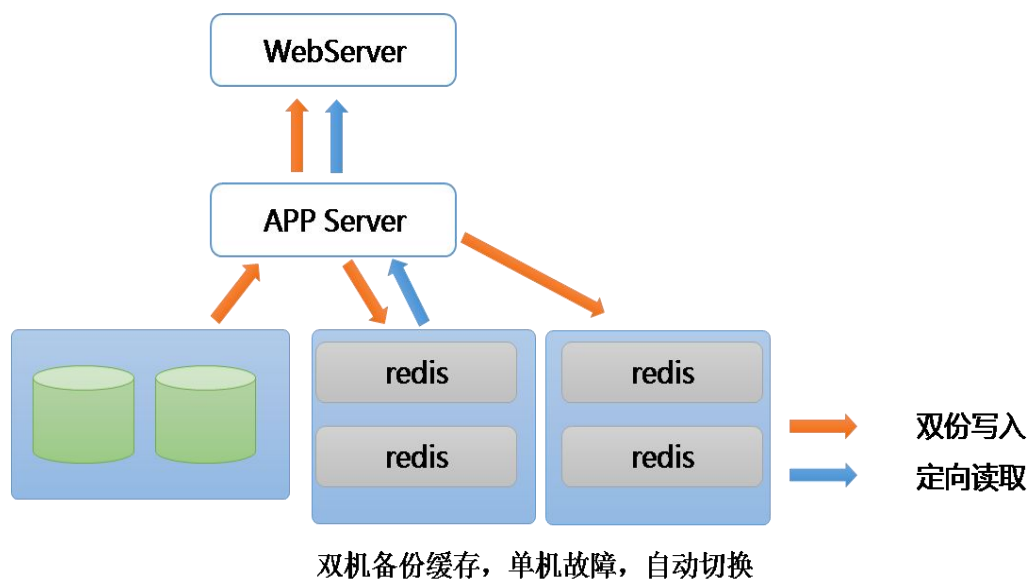


图 4.11 缓存服务器架构

统一缓存服务是为了保证互联网应用大并发和大流量情况下通过缓存服务降低服务对数据库服务器访问的压力，电商平台产品提供统一的缓存处理服务，采用 Redis 分布式缓存产品，提供整个电商平台的服务器端缓存应用服务。

当服务需要进行缓存时，可以通过访问代码直接完成缓存的存放和更新，缓存的读取对于用户来说时透明的，平台已经完成了对缓存服务的读取服务，无须在编码时考虑对缓存的读取，降低了代码级对缓存服务应用的复杂度。

采用分布式缓存服务，可以很方便地通过增加服务器组来实现缓存服务的容量升级和性能升级。

4.2.2 系统持久化设计

Redis 为互联网金融服务平台提供了两种持久化模式：RDB 和 AOF，关于这两种模式的持久化原理，在第 3 章中我们也详细介绍过，那么定位到互联网金融服务平台时，我们采用哪种模式更合适呢？

由于 Redis 与 Oracle 数据库采用的是读写分离的技术架构模式，那么 Redis 作为读库来讲，不存在数据损失问题，而且 RDB 模式对系统性能的影响也远远小于 AOF，即使 Redis 不幸需要重启，RDB 模式的数据恢复速度也远快于 AOF，从以上三个方面来考虑，对于互联网金融服务平台来讲，采用 RDB 模式应该算是最佳的选择。

4.2.3 高可用设计

Redis 可以通过配置主从数据库 Master-Slave 实现数据的复制(主从数据库配置在不同的服务器上)。主数据库一般用来读写，从数据库用来读和备份数据。当发生写操作时自动将数据同步给从数据库，而从数据库一般是只读的，并接受主数据库同步过来的数据。一个主数据库可以连接多个从数据库，而一个从数据库只能连接一个主数据库，基本结构如图 4.12 所示：

Sentinel 是 Redis 用于管理多个 Redis 服务器 (instance)，实现监控、提醒、故障转移的分布式系统。其中一台运行 Sentinel，另外两台运行两个 Redis 服务，同时 Redis 服务上跑 Sentinel 服务。

集群最小单位：1 台 Sentinel、2 台 Redis，Redis(主)与 Redis(从)为 Redis 集群，本身实现数据同步。Sentinel 作为哨兵监测服务，监控 Redis 服务是否正常，并且在 Redis 服务发生故障时候，起到故障转移的作用(如 Redis(主)down 掉，自动推选出 Redis(从)作为主，并推送其 Ip 到客户端，使客户端可连接正确的 Redis 服务端 Ip 进行读写操作。

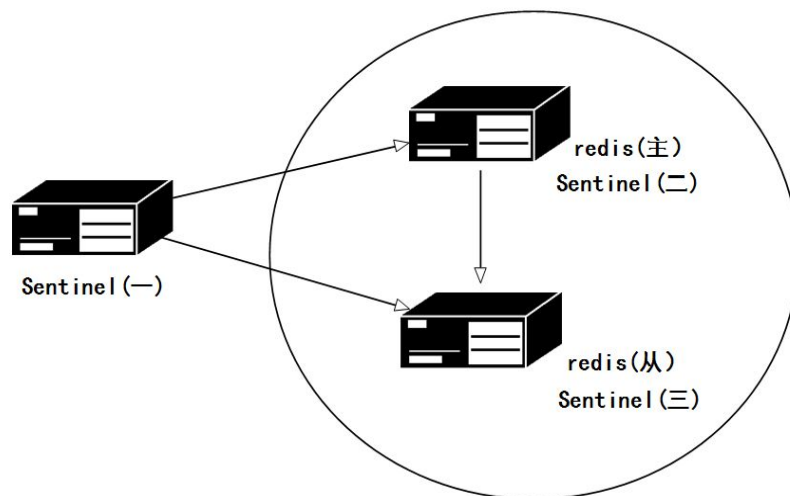


图 4.12 基本结构图

在 Redis 的各种操作中，持久化处理是一个相对耗时的操作。为了发挥 Redis 在互联网金融服务平台的最大性能，我们开启了 Master-slave 的主从复制功能，这样数据就能保持同步，但是我们关闭了主数据的持久化，这样主服务器就不会有其他的性能消耗，但同时一定要开启从数据库的持久化，这个很重要，不然一旦数据库崩溃，所有数据都灰飞烟灭了，影响也是巨大的。

在实现了主从同步和从数据库的持久化之后，我们对于数据库崩溃就能很从容处理：如果从数据库崩溃了，重启之后会自动从主数据库同步最新数据；如果主数据库崩溃了，则通过命令行把从数据库指定为主数据库后继续提供服务，同时原主数据库重启之后改为从数据库，则自动从原从数据库完成数据同步。

完成了 Redis 集群的部署，我们进行了如下场景的测试：

测试场景一：模拟 Redis（主）宕机，查看 Redis（从）是否可自动切换为主，并查看客户端是否可正常访问 Redis 进行读写操作。

测试方式：ps -ef|grep redis，找到该机器 redis 与 sentinel 进程号，执行 kill -9 操作。

测试结果：客户端正常访问 Redis（从）服务器，业务正常。

测试场景二：模拟 sentinel（一）宕机，查看客户端是否可正常访问 redis 进行读写操作。

测试方式：ps -ef|grep redis，找到该机器 sentinel 进程号，执行 kill -9 操作。

测试结果：客户端正常访问 Redis（主）服务器，业务正常。

测试场景三：模拟 redis（从）宕机，查看客户端是否可以正常访问 redis 进行读写操作。

测试方式：ps -ef|grep redis 找到该机器 redis 与 sentinel 进程号，执行 kill -9 操作。

测试结果：客户端正常访问 Redis（主）服务器，业务正常。

测试场景四：telnet 连接到 redis（主），写入数据后，telnet 到 redis（从），确认数据可同步完成。

测试方式：连接到 redis（主），写入数据：

```
[redis@ynetdev4 ~]$ telnet 192.168.65.8 6379
```

```
Trying 192.168.65.8...
```

```
Connected to 192.168.65.8.
```

```
Escape character is '^]'.
```

```
set key abcde
```

```
+OK
```

#使用 ctrl +] 后，输入 quit，退出 telnet 界面

连接到 redis（从），读取数据：

```
[root@ynetdevvs1 ~]# telnet 192.168.65.8 6379
```

```
Trying 192.168.65.8...
```

```
Connected to 192.168.65.8.
```

```
Escape character is '^'.
```

```
get key
```

```
$5
```

```
abcde
```

```
#使用 ctrl + ] 后，输入 quit，退出 telnet 界面
```

测试结果：Redis 主从服务器同步正常。

4.2.4 容量管理设计

互联网金融服务平台持久化设计中，我们选择了 RDB 的持久化模式，因此 RDB 文件会随着数据量的增大和时间的推移而不断变大，因此我们需要考虑如何进行容量管理？不过容量管理跟选择哪种持久化模式没有关系，因为即使使用 AOF 模式，这种模式即使提供了 rewrite 机制，把大于预定阈值的指令记录文件进行压缩处理，从而暂时避免了文件过大导致文件夹空间不足的风险，但是从长远来说，隐患依然是存在的。

由于 Redis 目前数据量并不大，我们给定快照文件足够的磁盘空间，同时我们将此快照文件夹纳入监控管理，一旦当前文件夹空间超过 70%，则会收到空间不足的风险报警。

4.2.5 Redis 安装及配置

表4.1 缓存服务器需求表

准备项	详细项	备注
服务器	三台	最小单元，无单点
用户	redis	可以是其他应用用户
目录	/home/redis	目录属主为 redis
安装介质	redis2.8.11.tar.gz	
依赖包	gcc 与 tcl	需 root 用户安装

根据我们对系统内缓存架构设计，我们在进行实施前，需要设定具体的缓存服务器需求表，如表 4.1 所示，根据需求表内容，我们准备相应的软硬件配置、配置操作系统用户及权限目前，为下一步的 Redis 安装配置做准备。

安装介质：redis-2.8.11.tar.gz；安装过程及步骤如表 4.2 所示：

表4.2 安装步骤表

```
#以下命令使用 redis 用户执行，将安装介质 copy 到安装目录，此处为
# /home/redis/，可按项目规划不同更改路径
cd /home/redis/
tar xvf redis-2.8.11.tar.gz
cd redis-2.8.11
make test
make PREFIX=/home/redis/ install
make && make install
#make && make install 将会把 redis-server、redis-cli、redis-benchmark、
#redis-check-aof、redis-check-dump 五个文件复制到 /usr/local/bin/下
```

配置过程及步骤如表 4.3 所示：

表4.3 配置步骤表

```
#将安装目录的配置文件 copy 到/etc/下
cp /home/redis/redis-2.8.11/redis.conf /etc/
cp /home/redis/redis-2.8.11/sentinel.conf /etc/
#redis(主)与 redis(从)修改以下
vi /etc/redis.conf
#修改 daemonize no
daemonize yes
#修改日志存储位置
logfile "/home/redis/logs/redis.log"
#修改 dbfile 目录,由编译文件目录更改为以下目录
```



```

dir "/home/redis/db"

#三台均修改以下配置文件

vi /etc/sentinel.conf

#将 127.0.0.1 的 ip 修改为 redis 主从中的任意一个即可

sentinel monitor mymaster 192.168.65.8 6379 1

/*这一行配置指示 Sentinel 去监视一个名为 mymaster 的主服务器，这个主服务器的 IP 地址为 192.168.65.8，端口号为 6379，而将这个主服务器判断为失效至少需要 1 个 Sentinel 同意（只要同意 Sentinel 的数量不达标，自动故障迁移就不会执行）。*/

#启动 redis 服务

/home/redis/bin/redis-server /etc/redis.conf

#启动 sentinel 服务

cd /home/redis/bin

nohup /home/redis/bin/redis-server /etc/sentinel.conf --sentinel &

```

从 redis 服务器上需要修改/etc/redis.conf 中添加

Slave of 10.192.32.61 6379

Redis 服务启停操作命令如表 4.4 所示：

表4.4 启停命令表

```

启动 redis:

/home/redis/bin/redis-server /etc/redis.conf

启动 sentinel:

cd /home/redis/bin

nohup /home/redis/bin/redis-server /etc/sentinel.conf --sentinel &

停止 redis:

```

```
/home/redis/bin/redis-cli -h 192.168.65.58 -p 6379 shutdown
```

停止 sentinel:

```
/home/redis/bin/redis-cli -h 192.168.65.58 -p 26379 shutdown
```

其他配置参数及用法如表 4.5 所示:

表4.5 参数解读表

参数	描述
daemonize	是否以后台 daemon 方式运行
pidfile	pid 文件位置
port	监听的端口号
timeout	请求超时时间
loglevel	log 信息级别
logfile	log 文件位置
databases	开启数据库的数量
rdbcompression	是否使用压缩
dbfilename	数据快照文件名（只是文件名，不包括目录）
dir	数据快照的保存目录（这个是目录）
appendfsync	appendonlylog 如何同步到磁盘（三个选项，分别是每次写都强制调用 fsync、每秒启用一次 fsync、不调用 fsync 等待系统自己同步）
appendonly	是否开启 appendonlylog，开启的话每次写操作会记一条 log，这会提高数据抗风险能力，但影响效率
save * *	保存快照的频率，第一个*表示多长时间，第三个*表示执行多少次写操作。在一定时间内执行一定数量的写操作时，自动保存快照。可设置多个条件

4.2.6 优化后的应用流程场景

我们已经完成对平台架构中缓存服务器的改造，又通过实际安装配置 Redis 服务器完成了基础工作准备，接下来我们在回顾一下之前提到的重要流程在 Redis 应用后优化。

(1) 客户注册流程：客户通过互联网金融服务平台进行平台客户注册，客户需要录入信息包含手机号码、图形验证码、手机验证码、邮箱、设置登录密码并阅读电商服务平台注册条款信息才能完成注册。注册过程中需要验证客户发送手机验证码，平台应用会生成一个随机的验证码信息，同时把验证码信息推送至 Redis 服务器端，Redis 创建一个 Key-YZM 来存储相应的手机及验证码信息：YZM: 186XXXX1245: 534963，为了满足验证码 5 分钟内有效的安全需求考虑，我们给 Redis 这个 Key-YZM 设置一个生命周期：Expire YZM 300 Seconds，当这个 Key 值过期时，Redis 会自动执行删除操作，生命周期内，Web 应用可以通过 Get 命令获取对应的 Value 值。优化后的用户注册流程如图 4.13 所示：

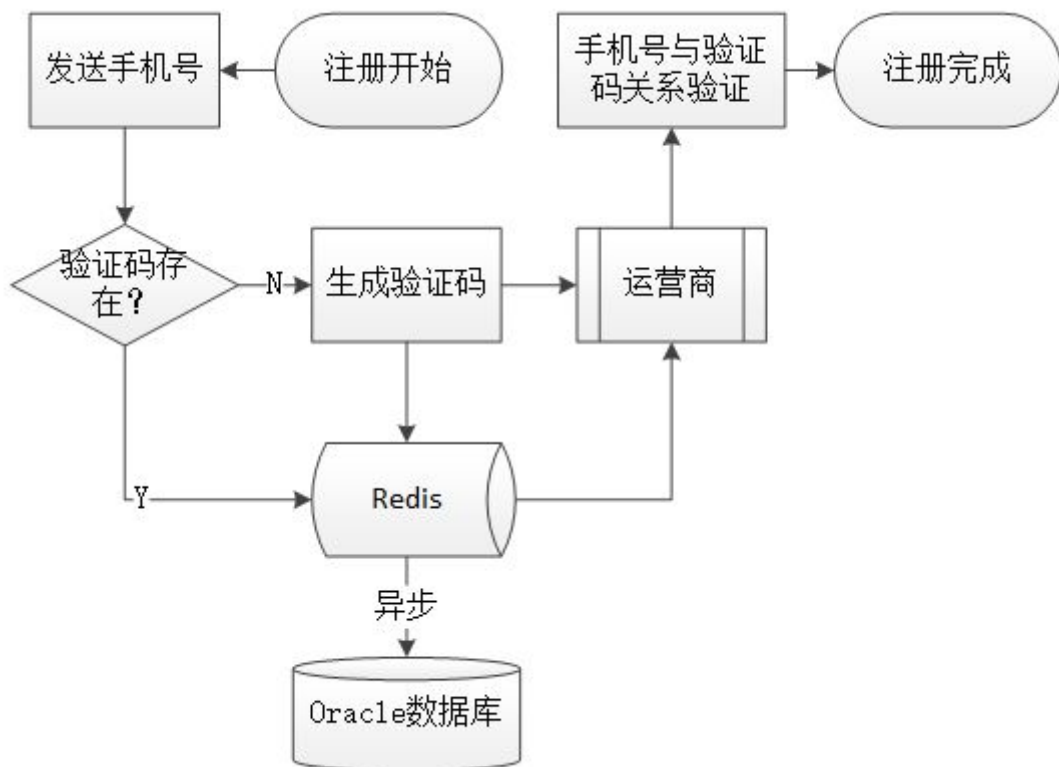


图 4.13 优化后的用户注册流程图

(2) 电商平台商品类目处理流程：为了满足大并发量的电商门户网站进程访问，我们改造了之前通过 Web 应用直接访问 Oracle 数据库商品类目配置表的方式，通过三种方式实现类目的初始化和更新操作：

1、当 Redis 服务器启动时，指定 Oracle 数据库的商品类目配置表数据 SET 到 Redis 的 Product Category_Key；

2、当电商平台管理人员通过管控系统修改商品类目后，管控系统调用 DEL 命令来删除 Product Category_Key 以及重新加载 Product Category_Key 的 Value 值；

3、应用设置定时任务，每天晚 9:00 调用 GETSET 命令获取 Oracle 数据库商品类目配置表最新的数据记录覆盖 Product Category_Key 的 Value 值。

通过把 Oracle 数据库商品类目配置表记录加载到 Redis 的操作，保证 Redis 中的商品类目时刻保持最新，这样在用户访问电商门户首页时，就不需要访问 Oracle 数据库，通过对 GET Product Category_Key 的 Value 值，就能完成商品类目的读取动作。

(3) 商城订单流程：客户通过电商门户，发起涉及多个商户的整体订单，

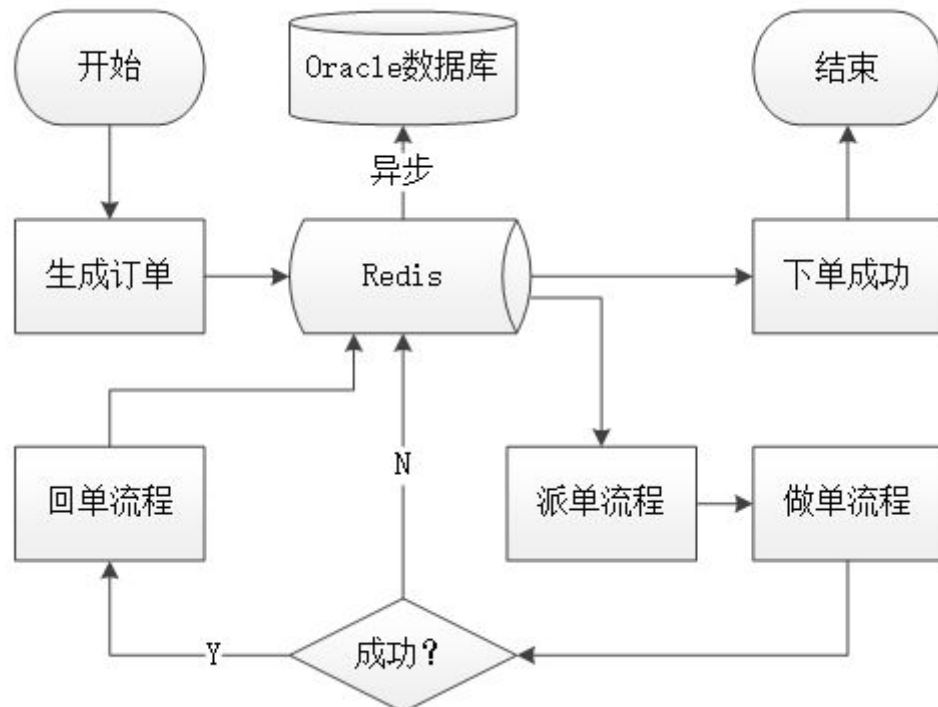


图 4.14 优化后的下单流程图

系统将订单信息登记在数据库的订单信息表，应用有专用进程扫描订单状态，对已下单的订单进行派单操作，同时修改订单状态为已派单，订单派单之后，会有相应应用完成做单流程，做单流程及发起与商户应用的交互，确认商户是否能满足客户的该订单信息，若商户能满足，则做单成功，调用回单流程给数据库修改订单状态为下单成功，同时返回客户端下单成功消息，在做单过程中，若该笔订单涉及商户为 1 至 2 个，则一次就能完成做单流程，若订单设计商户数目较多需要跟多家商户轮询通讯，而且通讯过程有可能由于网络等问题造成与商户的通讯超时，直接导致一次做单无法完成的情况，因此一次做单不成功的订单，又会返回数据库进行重新派单操作，循环处理直至做单成功。优化后的流程如图 4.14 所示。

4.3 本章小结

本章通过对互联网金融服务平台的架构分析和重要流程剖析，发现了当前系统存在的瓶颈问题是业务流程全部依附于对 Oracle 数据库服务器的访问，而 Oracle 数据库面对大数据量和高并发访问时，响应效率会很低，我们提出了依附于 Oracle 数据库服务器的架构设计不适用于互联网应用的本质问题，结合我们第 3 章对 Redis 技术研究后得出了 Redis 适合高并发的互联网应用的特性，契合前者的缺陷和后者的优势，我们对互联网金融服务平台技术架构做出相应调整，在支撑系统增加了缓存服务器，通过对 Redis 安装配置初始化，完成系统的持久化和高可用，并且对重点流程做出了相应的缓存处理流程调整，使平台更适应高并发的访问，保证了平台的实时性、响应速度和并发访问量。

5 测试及效果分析

通过对系统总体架构分析和系统重要流程剖析，我们已经发现系统的瓶颈问题主要在所有的业务流程都需要与关系数据库进行交互，那么当大数据量和大并发量的交易场景下，数据库由于 I/O 读写问题，造成系统整体性能的降低，直接表现为系统响应过慢，影响客户的使用体验。当然这中间还涉及到硬件配置、软件算法等其他影响要素，但是缓存设计的缺失应该是互联网金融平台的败笔之处。

通过对系统总体架构的研究后，发现了系统的瓶颈问题是关系数据库，由于关系数据库必须完成对磁盘的读写，所以仍然坚持使用关系数据库的模式，对系统的效率提升是有很多局限性的，并且关系数据库的模式也无法应付互联网应用的大数据量和高并发访问的场景，因此如何提升互联网金融服务平台的数据库层面设计成为提升应用性能的关键。通过对 Redis 缓存技术的深入剖析和研究，结合互联网金融服务平台的业务流程特点，我们决定将 Redis 缓存技术引入互联网金融服务平台，利用 Redis 缓存技术特性，完成流程改造和高可用设计等一系列优化方案。

本章在前几章节的理论研究基础之上，首先为了验证 Redis 缓存技术的架构设计性能要远高于 Oracle 数据库的架构设计，同时需要寻求最适合 redis 性能最大化的环境配置，我们先进行了性能测试阶段，为优化后的互联网金融服务平台生产环境搭建打基础做准备，有力性能测试的结果，我们就可以顺利搭建生产环境，随后在生产环境监测优化后的系统性能是否真的大幅提升。

5.1 性能测试

为了了解 Redis 缓存技术在互联网金融服务平台应用后的性能发挥情况和寻求平台性能最大化的环境及参数配置，我们先搭建测试环境进行性能验证性测试。因为目前还没有一个标准的 Redis 数据库性能测试的方案，同时互联网金融服务平台的业务流程测试也没有标准流程，所以我们就选取了相对复杂的商城订单流程进行性能部分测试。为了尽量精确、客观地测试 Redis 内存数据库的性能，我们分别模拟定量数量订单量、单线程、多线程三种情况下向 Redis 内存

数据库执行 SOL insert 操作，在相同的测试环境下，对比 Oracle 数据库进行相同数据测试。从验证测试结果来看，Redis 的速度是非常快的。测试环境配置及测试内容如表 5.1 所示：

表5.1 测试说明

项目	描述
测试情景	模拟商城订单生成量为10000和100000条记录，执行插入到redis内存数据库和Oracle数据库，测试整个过程的耗时情况（Linux i686环境）
测试用表	测试用主表tMailord和两个附表（tProduct表和tMerord表），主表有20个属性字段，其中13个非空字段都需要进行SQL Insert语句操作
测试操作	SQL Insert
测试数据结构	Redis是key-value型的内存数据库，支持原子操作，每次都必须操作数据库，根据Oracle测试主表tMailord表结构进行redis数据结构设计，采用String类型，“表名：字段名：字段值”的形式作为key，对应的结果值为value，redis数据插入采用mset命令

表5.2 线程数量-耗时

线程数量	Redis耗时（ms）	Oracle耗时（ms）
1	112591	785522
5	43473	221018
10	29767	142324
20	18953	122387
50	14653	119755

在现有测试环境中，硬件配置保持不变，只调整软件相关配置属性来进行测试。通过变量控制法对 2 个变量(线程数量，商城下单数量)进行性能测试。从测试的结果我们可以发现，线程数并不是越多越好的，商城订单的子单明细数

量越多，下单成功所用的时间越长，增加了 Redis 缓存设计之后的系统在整个性能上还是得到了大幅的提升。

测试场景 1，定量：工单总数量：100000，测试变量：线程数量，测试耗时数据如表 5.2 所示。

相同数量订单情况下，不同的线程数量插入的耗时折线图，如图 5.1 所示。

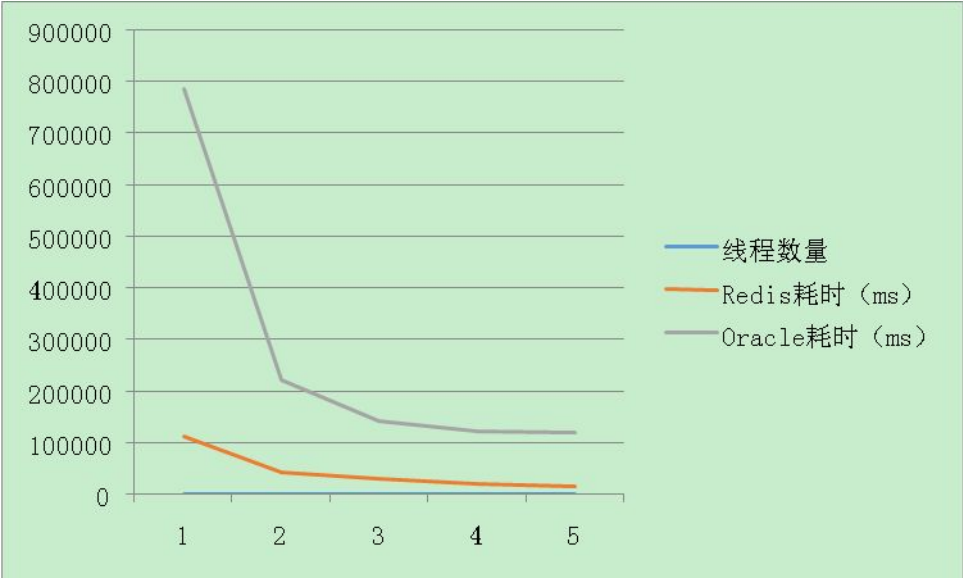


图 5.1 线程数-耗时

从线程数-耗时折线图的线性趋势可以看出折线表现“V”字型，表明多线程的订单处理效率是单线程处理效率好几十倍，但也说明线程并不是越多越好，当同时在线线程数量过多时，线程间的资源竞争将可能成为制约系统处理效率的关键因素，从而影响了处理效率。

表5.3 商城订单数量-耗时

订单数量	Redis耗时（ms）	Oracle耗时（ms）
10000	3720	14127
50000	15291	75332
100000	30315	141344
200000	62500	292152

从 Oracle 和 Redis 两条折线的位置我们可以得出这样的结论：相同线程数、相同订单处理数量的条件下，Redis 缓存架构比直连 Oracle 架构的商城订单处理

效率要高出很多倍，从这个流程来看，系统性能确实得到了很大幅度的提升，同时 Redis 对于多线程的并发处理表现也很稳定，在实际配置中，我们可以结合多方面情况，配置并发线程数。

测试场景 2，测试定量：线程数 10 个，测试变量：商城订单数量，耗时数据如表 5.3 所示。

商城订单数量的变动对系统耗时的影响如图 5.2 所示。

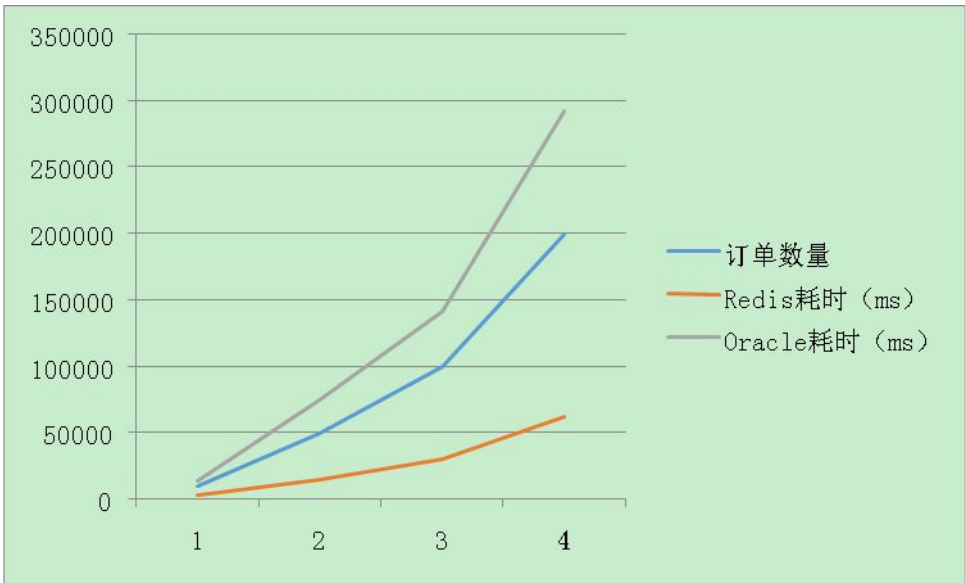


图 5.2 商城订单数量-用时

从上图可以看出，系统耗时与商城订单数量成正比关系。这和我们主观的判定是一致的，当商城订单数量越多的情况下，系统耗时越久。

通过上述变量控制法结合两个测试场景，对 Redis 完成了性能测试工作，测试的结果表明 Redis 对于 SQL 的执行速度是非常快的，引用了 Redis 缓存技术的互联网金融服务平台的整体性能方面也得到了很大的提升。

5.2 效果分析

Redis 缓存技术在互联网金融服务平台的应用研究到这里基本告一段落，上面我们通过搭建测试环境 Redis 性能和 Oracle 性能进行了比对，为了更进一步验证优化后的互联网金融服务平台的性能效果，我们对真实生产环境进行了 Redis 相关配置部署，希望通过在真实环境的试运行效果与改进前的数据做一个直接的对比。生产环境配置及运营数据说明如表 5.4 所示：

表5.4 生产环境配置及运营数据说明

环境配置及场景说明		Oracle 系统架构	Redis 缓存架构
生产环境硬件配置	CPU	四个物理 CPU，每个 CPU 包含 12 个核每核 2.20GHz	四个物理 CPU，每个 CPU 包含 12 个核每核 2.20GHz
	内存	200G	200G
	网卡速率	100Mb/s	100Mb/s
软件配置	服务器进程数	1 个 Oracle 进程	3 个 Redis 进程
	Nginx 配置端口	3 个	3 个
	派单线程数	5 个	5 个
	收单服务进程数	3 个，每个收单服务一个线程	3 个，每个收单服务一个线程
	流程回单进程数	45 个，其他服务进程数 30 个	45 个，其他服务进程数 30 个
场景 1	用户注册流程	单用户注册验证码申请到验证完成 50 秒，一分钟并发完成注册用户数量 5 个	单用户注册验证码申请到验证完成 20 秒，一分钟并发完成注册用户数量 20 个
场景 2	门户类目展示	单用户登录查询类目耗时 5 秒，10 用户并发登录耗时 70 秒。	单用户登录查询类目耗时 300 毫秒，10 用户并发登录耗时 3 秒。
场景 3 商城下单	生成订单数量	4324 个/分钟	13607 个/分钟
	做单数量	3663 个/分钟	96158 个/分钟
	流程回单数量	3023 个/分钟	91644 个/分钟
	回单数量	234 个/分钟	9162 个/分钟
资源消耗	内存占用	80GB 左右	在不开持久化 AOF 的情况下使用 40GB 左右
	CPU 使用率	80%左右	50%左右
结论	从上面的数据可以看出，我们在相同的硬件条件下，设定相同的应用场景，对时限或者用户数有严格限制的情况下，不管是从系统业务处理效率还是从系统的硬件资源消耗来讲，引入了 Redis 缓存技术的新平台架构设计的系统性能比原 Oracle 数据库架构设计下的系统性能要高出好几个数量级，达到了我们引入缓存技术后系统性能需要有质的飞跃的预想，此次研究的应用对于互联网金融服务平台有着很重大的意义，利用课题的学习和研究，解决了工作中的应用难题，应用实践的同时也加深了我们对研究技术的深刻理解和认识		

通过上表我们可以看出：在同等的硬件条件下，我们挑选了 3 个具有代表性的业务场景，通过对 Redis 应用前后的运营数据做对比，我们可以看出 Redis 的应用对各业务场景处理速度有了 30 倍的效率提升，并且通过硬件资源消耗参数可以看出，Redis 技术架构对硬件资源消耗也相对较低，那么我们可以得出这样的结论：Redis 的应用不仅节约了系统硬件资源消耗，同时对系统处理效率有了大幅的提升，针对场景 1 和场景 2 数据，我们可以看出对实时性和高并发也有了很大的提升，完成了我们引用 Redis 产品对互联网金融服务平台性能进行大幅提升的既定目标。

5.3 本章小结

本章先对改进后的互联网金融服务平台性能进行测试，获得发挥系统性能的运行环境，为实际运营的配置做好准备。之后在实际运营环境下，对改进后的互联网金融服务平台进行试运行。测试结果表明，虽然与管理部门的要求仍有一定的距离，但是使用了 Redis 的互联网金融服务平台性能得到了大幅的提高。

6 总结与展望

6.1 工作总结

本文以 Redis 缓存技术研究及在互联网金融服务平台为主线，采用了引入 Cache 技术概念和时下流行的 Cache 技术的介绍来突出我们系统内主要采用的 Redis 技术的思路，通过剖析 Redis 缓存技术的特性以及与同类产品性能对比体现出 redis 的优势。概要介绍应用背景互联网金融服务平台的当前架构设计及缺陷，分析出系统存在的瓶颈问题为没有缓存模块设计，结合系统架构、系统瓶颈和 Redis 研究成果，将 Redis 缓存技术引入到互联网金融服务平台的应用架构中，对系统架构和主要流程做了缓存模块设计及流程优化处理，大大提升了系统对外的服务能力。

Redis 内存数据库缓存技术一种快速有效的解决基于 Internet 的应用访问压力的重要技术之一，它是互联网金融服务平台优化的亮点内容，我们围绕 Redsi 的几个特性：速度快，使用 C 编写，可将所有数据加载到内存中；数据结构服务器，支持 5 种数据结构；持久化，支持数据持久化存储；支持主从复制；支持缓冲自动过期。逐步认识和熟悉 Redis 技术，并结合互联网金融服务平台自身的特点，将 Redis 技术巧妙的应用到互联网金融服务平台中。

最后我们对改造后的应用系统进行性能测试及实际环境测试，通过 redis 技术应用前与应用后的成果对比，体现出 Redis 缓存设计的应用给互联网金融服务平台带来的大幅提升。

6.2 工作展望

本文针对互联网金融服务平台应用下所有业务流程都直接与 Oracle 数据库直接交互的架构设计，提出了造成互联网金融服务平台对外服务能力低下的问题，为了解决这个瓶颈问题，我们深度研究了 Redis 缓存技术的特性以及与同类产品的对比，通过对互联网金融服务平台架构的分析和对 Redis 缓存技术剖析，为解决互联网金融服务平台响应速度及压力问题提供了一套解决的思路。但是由于这个应用缓存设计思路还有待进一步完善，对于系统的查询策略、更新策略、淘汰策略等内容没有更深入的探讨，而且当前互联网金融服务平台有部分

功能仍在开发，缓存模块设计思路的实现只对目前投产的流程做了相应改造，由于环境缺乏多样性数据和服务器性能制约等缘故，我们下一步就要考虑已投产功能和未投产功能流程做一次全面的梳理工作，既然平台目前已经存在缓存模块设计，那么就要从应用角度考虑如何应用缓存技术来提升服务流程，达到整体平台应用的服务全面提升。

（1）Cache 技术涉及的查询策略、淘汰策略等需要进一步完善和升级。

（2）互联网金融服务平台已投产流程及待开发流程都需要重新梳理，考虑缓存技术优化。

（3）生产环境目前用户数据结构单一，数据复杂度不足，未来在金融服务平台全面投产之后，数据量剧增的环境下，根据实际情况考虑缓存设计的完善。

参考文献

- [1]冯文唐,王纪梅.内存数据库关键技术研究[J].中国信息化,2013, 25(12):50-52.
- [2]连平.利率市场化:谁主沉浮[M].北京:中国经济出版社,2014.10-30.
- [3]中国人民银行.中国金融[J].中国金融出版社,2015,17(2):12-15.
- [4]王吉斌,彭盾.互联网+[M].北京:机械工业出版社,2015.14-23.
- [5]刘锋.互联网进化论[M].北京:清华大学出版社,2015.
- [6]曹玉林.基于 FastDB 的内存数据库实现机制研究[J].四川大学学报(自然科学版),2011,35(3):12-25.
- [7]刘云生.现代数据库技术[M].北京:国防工业出版社,2001.45-89.
- [8]汤庸等.高级数据库技术[M].北京:高等教育出版社,2005.65-103.
- [9]徐国风.实时数据库关键技术研究[D].中国优秀博硕士学位论文全文数据库(硕士).西安:西安建筑科技大学,2006.
- [10]梅巧玲等.内存数据库在互联网余票查询中的应用[J].铁路计算机应用,2014,23(3):41-44.
- [11]周岩.内存数据库在实时计费系统中应用的研究[D].北京:北京邮电大学,2012.
- [12]李岩.面向车联网的内存数据库设计与优化[D]. 大连:大连理工大学,2013.
- [13]李昭原等.数据库技术新进展[M].北京:清华大学出版社,1997.235-247.
- [14]Abraham Silberschatz, Henry Ekorth, s. Sudarshan 等著,杨冬青,唐世渭等译.数据库系统概念[M].北京:机械工业出版社,2005.345-356.
- [15]Knuth D E. The art of computer programming. V01.3[M]. USA: AddisonwesleyPubCO, 1973. 17-59.
- [16] Fagin Ronald, Nievergelt Jurg, Pippenger Nicholas, et al. Extendible hashing a fast access method for dynamic files[J]. ACM Transactions on Database Systems(TODS), 1979, 23(10): 214-268.
- [17]Larson Per-Ake. Linear hashing with partial expansions[A]. Proceedings of VLDB[C]. Saratoga, California, USA: VLDB Endowment, 1980. 235.
- [18]Lu H, Ng YY, Tian Z. Main memory database index structure revisited[C]. Australasian: 11th Australasian. IEEE, 2000. 65-73.
- [19]杨朝辉.主存数据库索引机制的研究与改进[D].南京:南京航空航天大学,2011.

- [20]Aho AV,Hopcroft JE.Design & Analysis of Computer Algorithms[M]. USA:A-ddi son-Wesley Pub CO,1974.133-245.
- [21]Comer Douglas.Ubiquitous B-tree[J].AEM Computing Surveys(CSUR),1979,11(2):324-356.
- [22]DeWitt D J,Katz R H,Olken F,et al.Implementation techniques for main memory data-base systems[C].Boston:ACM,1984.45-127.
- [23]E Levy,Silberschatz A.Incremental recovery in main memory database systems[J].IEEE Transactionson Knowledgeand Data Engineering,1992,6(7):509-516.
- [24]R Gupta,Ilaritsa J,Ramamritham K,et al.Commit processing in distributed realti-medatabase systems[A].Real-Time Systems Symposium,1996,17th IEEE[C].India: Motilal Banarsidass(MLBD),1996.220-229.
- [25]Chamberlin D D,Boyce R.SEQUEL:A Structured English Query Language.Proceedingsof ACM SIGMOD Workshop on Data Description.Access and Control[C].New York,USA:ACM,1974.235-256.
- [26]刘彩苹,李仁发,刘喜苹.面向嵌入式数据库的改进 B+树索引机制[J].计算机工程与科学,2007,29(1):101-105.
- [27]Gruenwald L,Huang J,Dunham M H,et al.Recovery in main memory databases[D]. Dallas,Texas,USA: University of Oklahoma,1996.
- [28]潘国浩.一种新的内存数据库快速日志恢复技术[J].计算机与信息技术,2007,43(5):79-81.
- [29]Kumar V,Burger A.Performance measurement of main memory database recovery algorithms based on update-in-place and shadow approaches[J].Knowledge and DataEngineering,IEEE Transactions on,1992,4(6):567-571.
- [30]张胜兰.内存数据库及其对外接口[D].济南:山东大学,2011.
- [31]肖红玉,陈海,黄静.Key-Value 数据库的研究与应用[J].自动化与仪器仪表,2010,9(2):87-89.
- [32]卢冬海,何先波.浅析 NoSQL 数据库[J].中国西部科技,2011,10(2):15-16.
- [33]ZawodnyJ.Redis:Lightweight key-value store that goes the extra mile[J].LinuxMagaz ine,2009,13(5):79-103.
- [34]刘鑫.高性能网站构建实战[M].北京:人民邮电出版社,2012.156-227.

- [35]Seeger M,Ultra-Large-Sites S.Key-Value stores:a practical overview[J].Computer Science and Media,2009,32(9):23-43.
- [35]张景云.基于 Redis 的矢量数据组织研究[D].南京:南京师范大学,2013.
- [36]李子骅.Redis 入门指南[M].北京:人民邮电出版社,2013.33-52.
- [37]朱进.基于内存数据库 Redi S 的轻量级矢量地理数据组织[J].地球信息科学学报,2014,19(3):15-32.
- [38]廖钢,刘旭宁,徐波.基于 Redis+Greasemonkey+J2EE 技术的促销信息展示系统的设计与实现[J].软件导刊,2012,15(11):95-97.
- [39]唐城.Redis 数据库在微博系统中的实践[J].厦门城市职业学院学报,2012,21(9):3-4.
- [40]汪林林,马锐.用 OCI 开发 Oracle 数据库的方法[J].计算机应用,2003,12(23):68-71.

致谢

首先，感谢我的导师王黎明老师。在三年的研究生学习生涯中，王黎明老师为我在学习过程中遇到的难题答疑解惑，答辩论文准备阶段，王黎明老师耐心的启发拓展我的思维，积极的引导我，教我以工作时间经验为基础，总结实战经验，然后结合理论依据撰写论文内容。论文从课题选择、资料的收集、实验的设计，都在王黎明老师的指导下进行。王黎明老师治学严谨的态度、诲人不倦的作风、满腹经纶的学识等一些优良作风对我的学习起到了积极的引导作用，更是我不满足于当前工作岗位而继续追求新技术新知识的坚实动力。在此，向王黎明老师致以崇高的敬意和衷心的感谢。

其次，感谢我的家人、同学和同事，是他们在身边默默的支持和鼓励着我，和同学们一起学习的时光，是属于我们共同的美好记忆，我也从他们身上学到了很多是在我以后的工作生活中都终生受用的东西，写论文过程中有遇到问题和坎坷，我也是找同学同事积极的沟通交流，是他们给了我思路 and 方向，也给了我完成论文的信心。感谢所有支持我的朋友，你们的支持永远是我不断进步的最大动力。

最后，还要感谢所有评阅本文、参加本文答辩的各位专家和教授！

攻读学位期间发表的学术论文

- [1] 邱书洋.试论软件工程项目在商业银行中的管理与质量控制[J].
河南科技, 2014, 548(18):307.